Web Browsers as a Pervasive Accessibility Solution

Current capabilities, public evidence, and a roadmap for personalized accessibility in the user agent

Michael G. Paciello

Version 1.13 (October 2026). Work in progress — a review draft, not for distribution, and subject to peer review for accuracy. Figures and claims are sourced but not yet independently verified by reviewers.

This paper was prepared with the assistance of AI (Anthropic's Claude), used to locate and verify sources and to draft text. The author reviewed every statement and source.

© 2026 Michael G. Paciello. All rights reserved.

This paper may be shared in full for the purpose of review and may be quoted or cited with attribution. It may not otherwise be reproduced, republished, or distributed, in whole or in part, without the written permission of the author.

Contents

- Abstract
- Key findings
- 1. Introduction
- 2. Engine architecture and accessibility foundations
- 3. Engine and browser profiles
- 4. What browsers already do for users
- 5. Public evidence on consistency
- 6. Post-source accessibility
- 7. ARIA as a post-source remediation strategy
- 8. Regulatory context
- 9. Gap analysis
- 10. On-device AI and the engines' stated concerns
- 11. Roadmap for engine and browser teams
- 12. The business case
- 13. The challenge to the browser industry
- 14. Questions for discussion
- 15. Conclusions
- Appendix A. U.S. and European regulatory comparison

© 2026 Michael G. Paciello. All rights reserved.

- Appendix B. Evidence sources
- Disclosures
- About the author
- References

Abstract

People with disabilities meet the web through a browser. Every page they read has been parsed, styled, laid out, and exposed to assistive technology by a browser engine. This paper asks how far today's engines and browsers already go toward making content usable for each person, what public evidence shows about the consistency of that support, and what engineering work would let a user state their needs once and have every page adapted to them.

The paper examines the three engines that underlie mainstream browsing (Chromium's Blink, WebKit, and Gecko) and two emerging engines (Servo and Ladybird). It draws on vendor documentation and source code, W3C specifications, the Interop 2024 accessibility focus area, Web Platform Tests (WPT), W3C ARIA-AT results, PowerMapper screen reader tests, and MDN browser-compat-data. Where no public data exists, the paper says so and labels any judgment as the author's assessment.

Four findings stand out. First, the engines have converged on similar accessibility architectures and agree closely on how they compute roles and accessible names: in the Interop 2024 accessibility focus area, all four tested browsers finished the year at or above 99.7% in preview builds. Second, browsers already ship many user-side aids, including automatic image descriptions, live captions, reading modes with read-aloud, and color overrides, but each is vendor-specific, uneven across engines, and not driven by standardized user preferences. Third, agreement on what browsers compute does not yet mean that people using screen readers hear the same thing in every browser. The W3C ARIA-AT project tests exactly this: it takes correctly built reference examples of common controls, such as tabs, dialogs, and menus, and checks what JAWS, NVDA, and VoiceOver actually convey with specific browsers (at the Candidate stage, JAWS and NVDA with Chrome, and VoiceOver with Safari). Because the examples are reviewed reference implementations, a failure points to the browser, the screen reader, or the way they interact, rather than to the web author. Only 3 of its test plans have reached the Recommended stage, about half are still at the Candidate stage, none yet covers live regions beyond alerts, and results for composite widgets still differ widely between combinations. A new effort, the Accessibility Compatibility Dataset (ACD), is beginning to collect exactly this browser-and-assistive-technology support data for developer tools, though it is in early development. Fourth, no current law or standard requires browsers to provide personalized adaptation. Section 508 and EN 301 549 reach browsers mainly through procurement, and the European Accessibility Act does not list standalone browsers.

The paper proposes a 36-month roadmap divided between shared engine work and browser-product work, with deliverables tied to public test suites. Under stated assumptions (engineering rates derived from published compensation data; staffing and engine shares estimated by the author), the estimated cost is about $17 million to $50 million per engine-and-browser pair at large-company engineering rates (base case about $32 million), and roughly $9 million to $15 million at non-profit rates. Engine work landed once in a shared engine is available to every browser built on it. The base-case annual cost, about $10.6 million, is under 0.004% of annual revenue for Alphabet, Apple, and Microsoft, about 1.0%

© 2026 Michael G. Paciello. All rights reserved.

to 1.6% for Mozilla, and many times Servo's current donation and grant income (Ladybird does not publish a budget).

The paper then makes the commercial case, built on three revenue pools that browser vendors already compete for. The first is AI agents: by Google's published guidance for Chrome, the accessibility tree that screen readers depend on is the primary way AI agents understand a page, other vendors' agent tools also read it, live-site agent success rates remain low, and forecasts of U.S. consumer spending through AI agents by 2030 range from $190 billion to $1 trillion. The second is search, advertising, and commerce from an aging population: Americans aged 50 and over accounted for 56 cents of every dollar spent in the United States in 2018, and disability rises with age. The third is per-seat enterprise and public-sector revenue, where accessibility is a condition of sale in government and education markets, subject to exceptions. Against the three largest reported revenue lines, a change of one-tenth of one percent would equal 10 to more than 26 times the base-case annual cost. The paper closes with a challenge to the browser industry: ten commitments, each measurable from public data, with dates from mid-2027 to the end of 2028.

Keywords: web accessibility; user agents; browser engines; accessibility tree; platform accessibility APIs; ARIA; WCAG 2.2; UAAG 2.0; EN 301 549; Section 508; European Accessibility Act; personalization; WAI-Adapt; Interop; Web Platform Tests; ARIA-AT

Key findings

- **Engines agree on computed semantics.** Interop 2024 included more than 1,000 tests of accessible name and role computation (Simmons, 2025). By the end of 2024, Chrome, Edge, and Firefox preview builds passed 100% and Safari Technology Preview passed 99.7% (Web Platform Tests, n.d.-a).
- **The architectures have converged.** Chromium and Firefox answer assistive technology requests from a cached copy of each page's accessibility tree held in the browser process, and WebKit has been developing an "isolated" tree, readable off the main thread, since 2019.
- **Screen reader output still varies.** In PowerMapper's December 2025 tests, overall reliability ranged from 80% (JAWS with Edge; VoiceOver on iOS with Safari) to 92% (NVDA with Firefox). W3C ARIA-AT has 3 test plans at the Recommended stage, all for links.
- **Browsers already help users, but not in a standard way.** All four major browsers ship a reading view; Chrome, Edge, and Firefox include read-aloud in it. Chrome and Edge offer opt-in automatic image descriptions for screen reader users. Only Safari exposes in-band audio-description tracks to web content by default, and no engine implements text audio descriptions (`<track kind="descriptions">`).
- **There is no portable accessibility profile.** Some settings sync within one vendor's account, but no browser imports or exports accessibility preferences for use in another browser, and the W3C WAI-Adapt work has had no new technical report publication since January 2023 (W3C, n.d.-a, n.d.-b).
- **Regulation reaches browsers indirectly.** Section 508 classes web browsers as platform software and EN 301 549 treats user agents as software, but both apply through procurement. The European Accessibility Act lists operating systems, not browsers. EN 301 549 V4.1.1, published 2

© 2026 Michael G. Paciello. All rights reserved.

September 2026, adds user-preference clauses for web pages (9.7) and software (11.7) and is awaiting citation in the Official Journal of the European Union.

- **The investment is modest for some vendors and not for others.** The base-case annual cost, about $10.6 million, is under 0.004% of annual revenue for Alphabet, Apple, and Microsoft (about 13 to 17 minutes of revenue), about 1.0% to 1.6% of annual revenue for Mozilla depending on the cost rate assumed, and many times Servo's current donation and grant income.
- **The potential return lies in three revenue pools worth hundreds of billions of dollars.** AI agents: Chrome's guidance calls the accessibility tree the primary way agents understand a page (Kulikowski & Hablich, 2026), and forecasts of U.S. agent-driven consumer spending by 2030 range from $190 billion to $1 trillion. Search and advertising: Alphabet earned more than $280 billion from online advertising in 2025, and Americans aged 50 and over accounted for 56 cents of every dollar spent in the United States in 2018. Per-seat enterprise and public-sector revenue: Microsoft reports more than 450 million paid Microsoft 365 commercial seats, sold into public-sector markets where accessibility is a condition of purchase, subject to exceptions.
- **A small change would repay the cost many times over.** One-tenth of one percent of Alphabet's advertising revenue, Microsoft's Productivity and Business Processes revenue, or Apple's Services revenue would equal about 10 to more than 26 times the base-case annual cost. For Firefox, the work would pay back if it retained or won about 2.1% of monthly active clients.
- **A measurable challenge.** Section 13 sets out ten commitments for the browser industry, each checkable from public data, due between June 2027 and December 2028.

1. Introduction

1.1 Why the browser layer

WCAG defines what accessible web content looks like, and decades of guidance, law, and litigation have been aimed at content authors. Yet the most recent WebAIM Million analysis found detected WCAG 2 failures on 95.9% of the top one million home pages, with an average of 56.1 detected errors per page, an increase over the previous year (WebAIM, 2026).

> **Author's assessment:** Content remediation alone has not produced an accessible web.

The browser sits between every author and every user. It already decides how content is rendered, how it is exposed to assistive technologies, and which user settings override author choices. The idea that the user's own agent should adapt content to the person is not new. It appears in early work on universal access, including the "personal interpreter" concept described in 1996 (Laux et al., 1996), and it is the premise of the W3C User Agent Accessibility Guidelines (UAAG 2.0; W3C, 2015) and of the W3C WAI-Adapt work. What has changed is capability: engines now build rich semantic trees, ship machine-learning features, and agree closely on how they compute semantics. This paper asks whether the browser can now carry a larger share of the accessibility load, and what it would take.

An estimated 1.3 billion people, about 16% of the world's population, experience significant disability (World Health Organization, 2022).

© 2026 Michael G. Paciello. All rights reserved.

1.2 Research questions

1. What do current browser engines and browsers provide, at the platform and user-interface level, that helps content become usable by people with disabilities?
2. What public, dated evidence shows how consistently that support works across engines and assistive technologies?
3. What engineering and standards work would let a user configure their needs once and have the browser adapt any content accordingly, and what would it cost?

1.3 Scope

The paper examines three engines that underlie essentially all mainstream browsing, and two emerging engines:

- **Chromium** (Blink rendering engine, V8 JavaScript engine), used by Google Chrome, Microsoft Edge, Samsung Internet, Opera, Brave, and Vivaldi, and by embedding frameworks such as the Chromium Embedded Framework (CEF), Electron, and Qt WebEngine.
- **WebKit** (WebCore, JavaScriptCore), used by Safari on macOS, iOS, iPadOS, and visionOS, and by the WebKitGTK and WPE ports on Linux.
- **Gecko** (with the SpiderMonkey JavaScript engine), used by Firefox on desktop and, through GeckoView, Firefox for Android. Firefox for iOS uses WebKit.
- **Servo**, a Rust-based engine that has been a Linux Foundation Europe project since September 2023, with Igalia leading development (Linux Foundation Europe, 2023).
- **Ladybird**, an independent engine (LibWeb, LibJS) developed by the non-profit Ladybird Browser Initiative, which has targeted 2026 for its first alpha release (Ladybird Browser Initiative, n.d.).

In StatCounter's worldwide data for September 2026, Chrome (66.36%), Safari (18.27%), Edge (6.04%), Firefox (2.80%), Samsung Internet (2.31%), and Opera (1.77%) together accounted for about 97.6% of page views (StatCounter, 2026). StatCounter counts browsers rather than engines, and it counts Chrome and Firefox on iOS as Chrome and Firefox even though they run on WebKit, so engine share cannot be read directly from these figures. Outside the European Union and Japan, Apple's App Store rules require browsers on iOS and iPadOS to use WebKit. Apple has allowed alternative engines for users in the EU since iOS 17.4 (iPadOS 18) and in Japan since iOS 26.2 (Apple, n.d.-a, n.d.-b). Apple's published criteria for those alternative engines include web-platform and JavaScript conformance thresholds but no accessibility requirement.

1.4 Method and evidence

Every factual statement in this paper was checked against a source read between 8 October 2026 and the date of release, with preference for primary sources: W3C specifications and test dashboards, browser vendor documentation and source code, regulatory texts, and company filings. Sources and access dates appear in the References. Three kinds of statement are distinguished:

- **Measured:** results from public test suites (Interop, WPT, ARIA-AT, PowerMapper) or compatibility data (MDN browser-compat-data, caniuse), cited with the date of the run or report.

© 2026 Michael G. Paciello. All rights reserved.

- **Documented:** features and behaviors stated in vendor documentation or visible in published source code.
- **Author's assessment:** judgments about significance, priority, or likely effort. These appear in separate, boxed notes headed "Author's assessment", so readers can always tell the author's interpretation apart from verified fact and weigh it accordingly.

The paper does not report new user testing. Its cost and return figures are estimates built from stated assumptions and public financial data, and Section 12 shows the arithmetic so readers can substitute their own.

1.5 Terminology: content conformance and user agent conformance

WCAG applies to web content (W3C, 2024). Browsers are not evaluated for WCAG conformance, although the browser's own interface can be, and Section 508 applies WCAG 2.0 to software user interfaces (U.S. Access Board, 2017, E207.2). The W3C guideline written for browsers is UAAG 2.0, which is a W3C Working Group Note from December 2015, not a Recommendation (W3C, 2015). The working group that produced it has closed, and W3C has said it does not plan to advance UAAG 2.0 and intends to include user agent considerations in future guidelines work (W3C WAI, 2024a). Several gaps identified in this paper correspond to UAAG 2.0 success criteria that were specified in 2015 but are not consistently implemented. Section 9 maps them.

2. Engine architecture and accessibility foundations

2.1 The accessibility tree

Each engine builds an accessibility tree from the Document Object Model (DOM) and computed styles. Each node carries a role, an accessible name and description, states, properties, and relationships. Native HTML semantics are mapped according to the HTML Accessibility API Mappings (HTML-AAM), ARIA semantics according to the Core Accessibility API Mappings (Core-AAM), and names and descriptions according to the Accessible Name and Description Computation (AccName) (W3C, 2026a, 2026e, 2026f).

Engines may omit, or mark as ignored, nodes that carry no semantics, such as a `div` or `span` with no role, name, or focusability. Their children remain exposed under the nearest included ancestor. The exact rules are engine- and platform-specific and are not fully specified. When a generic element is exposed, Core-AAM maps its `generic` role to platform roles such as UIA Group and AXGroup (W3C, 2026a). `role="none"` removes only an element's own semantics, and browsers ignore it on elements that are focusable or carry global ARIA attributes; `aria-hidden="true"` and `display: none` remove the whole subtree. Construction is lazy and incremental: Chromium, for example, builds accessibility objects on demand and sends batched updates after layout (Chromium Project, n.d.-a).

2.2 Platform accessibility APIs

Browsers expose the tree to assistive technologies through operating system APIs.

Windows. Microsoft Active Accessibility (MSAA) was introduced as a platform add-in for Windows 95 (Microsoft, n.d.-a); a December 1997 Microsoft message to the W3C WAI Interest Group describes the MSAA 1.1 redistributable and its inclusion in Windows 98 (Oppermann, 1997). Microsoft describes UI Automation (UIA) as a newer, more capable technology and continues to support MSAA for backward

© 2026 Michael G. Paciello. All rights reserved.

compatibility (Microsoft, n.d.-a). IAccessible2 (IA2) extends MSAA. It was developed by IBM and contributed to the Free Standards Group in December 2006; the Free Standards Group merged with OSDL in 2007 to form the Linux Foundation, which now hosts the specification (Updegrove, 2006; Linux Foundation, n.d.). Since Chrome 138 in 2025, Chromium has exposed native UIA by default, the result of a joint Microsoft Edge and Google effort, while keeping its MSAA and IA2 implementation (Beaudry, 2025). NVDA continues to use IA2 by default with Chromium; an NVDA maintainer has described Chromium's UIA implementation as weaker than its IA2 implementation (NV Access, 2025). Firefox implements MSAA and IA2 and also includes a native UIA provider (Mozilla, n.d.-a).

macOS. Browsers implement AppKit's NSAccessibility protocol, the "AX API" in W3C mapping specifications (Apple, n.d.-c).

iOS, iPadOS, and visionOS. Apps expose accessibility through UIKit's UIAccessibility, not NSAccessibility (Apple, n.d.-d). VoiceOver is common to all Apple platforms, but the APIs differ.

Linux. Chromium and Firefox implement ATK, which is bridged to the AT-SPI2 D-Bus protocol. WebKitGTK and WPE WebKit have used AT-SPI2 directly, without ATK, since 2022 (WebKit Bugzilla, 2022). Orca is the principal Linux screen reader. Sun Microsystems engineers, working with Mozilla, built much of Firefox's early Linux accessibility support in the mid-2000s (Mozilla, n.d.-b).

2.3 Converging architectures

Assistive technologies query platform APIs largely synchronously, while modern browsers render content in sandboxed processes. All three major engines have adopted, or are developing, similar approaches:

- **Chromium** builds the tree in the renderer, serializes batched updates to the browser process, and answers assistive technology requests from that cached copy without blocking on the renderer (Chromium Project, n.d.-a).
- **Firefox** re-architected its accessibility engine ("Cache the World") so the parent process holds a cache of every content process's tree. It shipped on Android in Firefox 102, on Windows and Linux in Firefox 112, and on macOS in Firefox 113 (May 2023); Mozilla reported improvements of up to 20 times in worst cases (Mozilla Accessibility Team, 2023).
- **WebKit** has developed an "isolated" accessibility tree since 2019, a read-only copy that macOS assistive technology requests can query off the main thread, with work continuing through 2025 (WebKit Bugzilla, 2019, 2025).

Author's assessment: This convergence matters for the roadmap. A cached, serializable tree in a privileged process is the natural place to apply user-preference-driven transformations and to repair semantics, because it is already where engines reconcile what content says with what assistive technology receives.

© 2026 Michael G. Paciello. All rights reserved.

2.4 Engines at a glance

Table 1. Browser engines examined in this paper (status as of October 2026)

Engine	Primary steward	Rendering / JS engine	Browsers and embedders	Accessibility API support	Notes
Chromium (Blink)	Google, with major contributions from Microsoft and others	Blink / V8	Chrome, Edge, Samsung Internet, Opera, Brave, Vivaldi; CEF, Electron, Qt WebEngine	MSAA, IA2, UIA (default since Chrome 138); NSAccessibility; ATK/AT-SPI; Android	3 of the 6 owners listed in `ui/accessibility/OWNERS` use microsoft.com addresses (Chromium Project, 2026b)
WebKit	Apple, with ports maintained by Igalia and others	WebCore / JavaScriptCore	Safari; all iOS browsers outside the EU and Japan; WebKitGTK; WPE	NSAccessibility (macOS); UIAccessibility (iOS, iPadOS, visionOS); AT-SPI2 directly (GTK, WPE)	Isolated accessibility tree in development since 2019
Gecko	Mozilla	Gecko / SpiderMonkey	Firefox (desktop and Android via GeckoView)	MSAA, IA2, native UIA provider; NSAccessibility; ATK/AT-SPI; Android	Parent-process tree cache since 2022–2023
Servo	Linux Foundation Europe; development led by Igalia	Servo / SpiderMonkey (via mozjs)	servoshell demo browser; embeddable Rust library published April 2026	Web-content accessibility through AccessKit, behind a preference (2026)	Grant-funded accessibility work, 2025–2026
Ladybird	Ladybird Browser Initiative (US non-profit)	LibWeb / LibJS	Ladybird (pre-alpha; alpha targeted for 2026 on Linux and macOS)	Screen reader support proposed (pull request #9156; merge status unconfirmed as of 9 October 2026)	Donation-funded; BSD-2-Clause license

Sources: Beaudry (2025); Chromium Project (2026b); WebKit Bugzilla (2019); Mozilla Accessibility Team (2023); Linux Foundation Europe (2023); Servo Project (2026a, 2026c, n.d.-b); Ladybird Browser Initiative (n.d., 2026).

3. Engine and browser profiles

3.1 Chromium

Blink builds the accessibility tree in the renderer process. Code in `content/renderer/accessibility` serializes batched tree updates to the browser process,

© 2026 Michael G. Paciello. All rights reserved.

where frame trees are combined and cached. Platform implementations derive from a common abstraction in `ui/accessibility/platform` (Chromium Project, n.d.-a, n.d.-b). Chrome DevTools can replace the DOM tree in the Elements panel with a full-page accessibility tree, and its Accessibility pane shows computed properties for each node (Google, 2026a).

Chromium also contains a `chrome.automation` API that exposes the accessibility tree to extensions. It is a private API restricted to an allowlist of extension IDs and Google component extensions, such as ChromeOS's ChromeVox screen reader; it is not available to ordinary Chrome Web Store extensions (Chromium Project, 2026a). Public extensions can inject user-origin CSS through `chrome.scripting.insertCSS` with `origin: "USER"` (Google, n.d.-a).

3.2 WebKit and Safari

On macOS, WebKit implements NSAccessibility; on iOS, iPadOS, and visionOS it implements UIKit accessibility. VoiceOver's rotor lets users move through web content by element type, using structure that WebKit exposes. Safari's Web Inspector shows, for a selected element, how it is exposed to assistive technologies, and its Audit tab includes a default accessibility audit (Apple, n.d.-e; WebKit Project, 2022). Safari is the only one of the four major browsers that exposes in-band alternate audio tracks, including audio-description tracks, to web content by default (`HTMLMediaElement.audioTracks`); Chromium and Gecko expose that API only behind flags (MDN, 2026). WebKit source code shows that Apple builds read the operating system's preferred audio characteristics when selecting tracks (WebKit Project, 2026); this is a reading of source code, not a tested result.

3.3 Gecko and Firefox

Mozilla's accessibility work was under way at Netscape by early 2001, with Netscape engineer Aaron Leventhal as its public lead, and Mozilla announced its first accessibility alpha, with MSAA support and keyboard access, in December 2001 (Leventhal, 2001a, 2001b, 2001c). In 2007 the Mozilla Foundation funded NV Access, the developer of NVDA, to improve support for rich web content in Firefox (Hecker, 2007). Firefox's Accessibility Inspector shows the full accessibility tree and can check pages for contrast, keyboard, and text-label issues, simulate color-vision deficiencies, and show tabbing order (Mozilla, n.d.-c).

3.4 Servo

Servo began exposing web-content accessibility in early 2026 using AccessKit, gated behind an `accessibility_enabled` preference. Each webview exposes its own tree, which the embedding application attaches to its interface tree; on Linux the result reaches Orca through AT-SPI, and incremental tree updates were added in mid-2026 (Servo Project, n.d.-b, 2026a, 2026b). Germany's Sovereign Tech Agency has invested €545,400 in Servo for 2025–2026, covering accessibility, embedding support, and maintainability (Sovereign Tech Agency, n.d.).

3.5 Ladybird

In January 2025, W3C staff member and Ladybird contributor Mike Smith reported that Ladybird was passing nearly all accessibility-related WPT tests (Faulkner, 2025). Those tests check computed roles and names, not platform API output. A pull request titled "Add screen-reader support: Orca and VoiceOver" proposes exposing a tree built in Ladybird's WebContent process through NSAccessibility on macOS and

© 2026 Michael G. Paciello. All rights reserved.

Qt's AT-SPI2 bridge on Linux (Ladybird Browser Initiative, 2026). As of 9 October 2026, its merge status could not be confirmed from the public page.

> **Author's assessment:** New engines are the clearest opportunity to design personalization into the architecture from the start, and also the most resource-constrained (see Section 12).

4. What browsers already do for users

Browsers now ship a substantial set of user-facing accessibility aids. Table 2 lists the main ones by browser. Features provided by the operating system rather than the browser are noted separately, because they are available to every browser on that platform.

Table 2. Selected built-in browser accessibility features, desktop unless noted (vendor documentation and source code, read 8 October 2026)

Feature	Chrome	Edge	Firefox	Safari
Automatic descriptions of unlabeled images for screen reader users	Yes, opt-in; images sent to Google; setting syncs with account	Yes, opt-in (Edge 97+ policy); Microsoft cloud service; includes text recognition	Drafts alt text only for images a user adds in its PDF editor, using an on-device model (Firefox 130+)	No browser feature; VoiceOver describes images at the operating system level
Live captions for audio the browser plays	Live Caption, on-device, opt-in; optional translation sends captions to Google	Windows only (Edge 103+); Microsoft states it is not generally available	None built in	No browser feature; Live Captions is an operating system feature
Reading view with spacing and theme controls	Reading mode	Reading mode (formerly Immersive Reader; Edge 149+)	Reader View	Reader (theme, font, and zoom)
Read aloud with voice choice	Yes, with word or sentence highlighting	Yes	Yes (Narrate)	Listen to Page on iOS 17+ (no Apple documentation found)
Caret browsing	Yes (F7)	Yes (F7)	Yes (F7)	No documented mode
Page zoom range	25%–500%	Chromium-based (range not separately documented)	20%–500% (Firefox 158 release branch, read 8 October 2026)	Not documented by Apple
Text-only scaling or minimum font size	Font size and minimum font size	Page zoom settings	Zoom text only; minimum font size	Minimum font size
Forced colors (Windows contrast themes)	Yes	Yes	Yes, plus its own cross-platform color override	Not implemented

© 2026 Michael G. Paciello. All rights reserved.

Feature	Chrome	Edge	Firefox	Safari
Native user style sheet	No (removed in Chrome 33, 2014); extensions only	No; extensions only	Yes, behind an advanced preference	Yes (macOS Settings > Advanced)
Enhanced focus indicator setting	"Show a quick highlight on the focused object"	"Show a high visibility outline on the focused object"	No equivalent setting found	No equivalent setting found
Page translation	Yes	Yes	Yes, on-device	Yes
Opt-in artificial intelligence (AI) summarize or explain	Gemini in Chrome	Copilot in Reading mode	Sidebar chatbot prompts	Apple Intelligence Summarize in Reader
Always underline links setting	No equivalent setting found	No equivalent setting found	Yes (Firefox 121+, December 2023): Settings > Accessibility > Motion and link styling > Always underline links	No equivalent setting found

Sources: Google (n.d.-b, n.d.-c, n.d.-d, n.d.-e, 2026b); Microsoft (2022, 2025a, 2026a, 2026b, 2026c, n.d.-b); Mozilla (2024a, 2026b, n.d.-d, n.d.-e, n.d.-f, n.d.-g); Apple (n.d.-f, n.d.-g); Chromium Project (2013, 2026c). Zoom ranges are taken from Chromium and Firefox source code. Availability of AI features varies by region, language, and account.

Several conclusions follow:

- **Repair already happens at the user's side.** Chrome and Edge describe unlabeled images on request, and Chrome runs text recognition on scanned PDFs for screen reader users (Google, 2024). These features do not change the page's markup, so authors remain responsible for text alternatives under WCAG 1.1.1.
- **Text-spacing control exists only inside reading views.** Users can adjust line, letter, and word spacing in Chrome, Edge, and Firefox reading views, but not on arbitrary pages without an extension or a user style sheet.
- **Audio description is the least supported media feature.** No engine implements text-based `<track kind="descriptions">`; each has an open implementation bug (MDN, 2026).
- **Developer tools are already substantial.** Chrome, Edge, and Firefox show full accessibility trees and simulate vision deficiencies, and Safari offers an accessibility audit. None of them previews what a screen reader would announce.
- **Each feature is configured separately, per browser.** No feature is driven by a standardized description of the user's needs.

© 2026 Michael G. Paciello. All rights reserved.

5. Public evidence on consistency

No public test suite rates browsers against WCAG, because WCAG applies to web content; a browser's own interface can be evaluated (Section 1.5), but that is not what these suites measure. What public suites measure is narrower: whether a browser implements a feature that authors rely on, what it computes and exposes to accessibility APIs, and what a particular screen reader announces with a particular browser. Tables 3 and 4 therefore report only measured or documented results, never a rating of a browser's "WCAG support".

5.1 Interop 2024 and Web Platform Tests

Interop is a cross-browser effort in which browser vendors agree on focus areas and track pass rates on shared tests. Accessibility was a scored focus area only in 2024. It covered computation of accessible names and roles for ARIA roles and HTML elements (Simmons, 2024). Accessibility testing has continued as an investigation in 2025 and 2026, aimed at expanding platform-API testing (Web Platform Tests, n.d.-c).

Table 3. Interop 2024 accessibility focus area scores (wpt.fyi data files, read 8 October 2026)

Run series and date	Chrome	Edge	Firefox	Safari	All four pass
Stable releases, 1 January 2024	82.8%	82.8%	80.3%	84.6%	75.7%
Preview builds, 30 December 2024	100%	100%	100%	99.7%	99.7%
Stable releases, 6 February 2025 (Chrome 133, Edge 132, Firefox 135, Safari 18.3)	95.6%	95.6%	95.6%	99.7%	95.3%

The WPT accessibility directories continue to be run daily. For the aligned stable runs of 6 October 2026 (Chrome 154, Edge 154, Firefox 157, Safari 27.0), subtest pass rates were: AccName, Chrome 96.5%, Firefox 95.5%, Safari 95.5%; WAI-ARIA, Chrome 98.8%, Firefox 96.9%, Safari 94.8%; HTML-AAM, Chrome 87.4%, Firefox 87.8%, Safari 94.9%. Edge matched Chrome within one subtest (Web Platform Tests, 2026).

These results measure the browser's internally computed role and name, through the WebDriver "Get Computed Role" and "Get Computed Label" commands. They do not measure what platform APIs expose or what screen readers say. A newer WPT test type, being developed by Igalia with NLnet funding, checks platform mappings for IA2, the macOS AX API, and ATK/AT-SPI; its coverage is still partial (Web Platform Tests, n.d.-b; NLnet, n.d.).

5.2 Assistive technology output

Two sources report what screen readers actually convey with specific browsers.

Why this matters. A browser can compute the right role and name and still fail the user, because what a person actually hears depends on how the browser passes that information to the operating system and how the screen reader interprets it. Readers wary of ARIA should note that ARIA-AT does not test whether web authors use ARIA well. It uses correctly built reference examples from the W3C ARIA Authoring Practices Guide, so a failure shows that even correct code is not yet conveyed consistently, which web authors cannot fix on their own.

© 2026 Michael G. Paciello. All rights reserved.

W3C ARIA-AT tests ARIA Authoring Practices Guide examples with JAWS, NVDA, and VoiceOver. As of 8 October 2026, 3 test plans had reached Recommended (all link examples, November 2025), 23 were Candidate, 4 were Draft, and 15 were in research and development, including combobox and landmarks. There is no test plan for tree or treegrid, and none for live regions other than `role="alert"`. Candidate reports cover JAWS and NVDA with Chrome and VoiceOver with Safari only, so they cannot compare Gecko with Chromium (W3C ARIA and Assistive Technologies Community Group, 2026). Selected Candidate results, which the project marks "Unapproved Report", are shown in Table 4.

PowerMapper publishes screen reader compatibility tests across about 195 test cases. Its December 2025 results report overall reliability of 81% for JAWS with Chrome, 80% for JAWS with Edge, 84% for JAWS with Firefox, 90% for NVDA with Chrome, 90% for NVDA with Edge, 92% for NVDA with Firefox, 88% for VoiceOver on macOS 15.7 with Safari 26, and 80% for VoiceOver on iOS 18.6 with Safari (PowerMapper Software, 2025). Image alt text and explicit form labels passed in every combination. PowerMapper rates labeling an `<audio>` element with `aria-label` at 1% reliability and with `aria-labelledby` at 0%, across the screen reader and browser combinations it tests, including older versions.

Table 4. Evidence relevant to each WCAG 2.2 guideline, by engine (feature support from MDN browser-compat-data and caniuse; assistive technology results from ARIA-AT and PowerMapper; read 8 October 2026)

WCAG 2.2 guideline	Chromium (Chrome, Edge)	WebKit (Safari)	Gecko (Firefox)	Caveats
1.1 Text Alternatives	`img alt` conveyed 100% with JAWS and NVDA (PowerMapper)	100% with VoiceOver on macOS and iOS (PowerMapper)	100% with JAWS and NVDA (PowerMapper)	Screen reader and browser measured together
1.2 Time-based Media	WebVTT captions since Chrome 23; `kind="descriptions"` not implemented	WebVTT since Safari 6; in-band audio tracks exposed by default; `kind="descriptions"` not implemented	WebVTT since Firefox 31; `kind="descriptions"` not implemented	Feature presence only; no public data on caption rendering quality
1.3 Adaptable	Minimal Data Grid: JAWS 75%, NVDA 83% (ARIA-AT, provisional)	Minimal Data Grid: VoiceOver 69% (ARIA-AT, provisional)	Not in ARIA-AT Candidate scope	No ARIA-AT data yet for landmarks or combobox
1.4 Distinguishable	`forced-colors` since Chrome 89, Edge 79; `prefers-contrast` since 96	`prefers-contrast` since 14.1; no forced colors mode	`forced-colors` since 89; `prefers-contrast` since 101	No public data for zoom, reflow, or text-spacing behavior
2.1 Keyboard Accessible	Tabs: JAWS 100%, NVDA 95% of required assertions (ARIA-AT, provisional)	Tabs: VoiceOver 66% (ARIA-AT, provisional)	Not in ARIA-AT Candidate scope	Screen reader interaction, not keyboard-only operation
2.2 Enough Time	No public browser-level data	No public browser-level data	No public browser-level data	Content responsibility

© 2026 Michael G. Paciello. All rights reserved.

WCAG 2.2 guideline	Chromium (Chrome, Edge)	WebKit (Safari)	Gecko (Firefox)	Caveats
2.3 Seizures and Physical Reactions	`prefers-reduced-motion` since 74	Since 10.1	Since 63	Feature presence only
2.4 Navigable	`:focus-visible` 86; `inert` 102; `popover` 114. Modal Dialog: JAWS 100%, NVDA 94% (ARIA-AT, provisional)	`:focus-visible` 15.4; `inert` 15.5; `popover` 17. Modal Dialog: VoiceOver 81% (ARIA-AT, provisional)	`:focus-visible` 85; `inert` 112; `popover` 125	No public data on focus-indicator rendering
2.5 Input Modalities	No public browser-level data	No public browser-level data	No public browser-level data	Mainly content requirements
3.1 Readable	`lang` reliably conveyed; `xml:lang` and language subtags not fully reliable (PowerMapper)	Same classification (PowerMapper)	NVDA with Firefox fails subtag tests (PowerMapper)	Depends on screen reader speech engines
3.2 Predictable	No public browser-level data	No public browser-level data	No public browser-level data	Content responsibility
3.3 Input Assistance	Date and time inputs fully supported since 25	Desktop lacks `week` and `month` inputs	Lacks `week` and `month` inputs	No data on exposure of validation messages
4.1 Compatible	Interop 2024: 100% (preview, end of 2024). `ariaNotify()` since 141, partial	Interop 2024: 99.7% (preview, end of 2024). `ariaNotify()` not supported	Interop 2024: 100% (preview, end of 2024). `ariaNotify()` since 150	WPT pass rates in Section 5.1

Sources: MDN (2026); Deveria (2026); W3C ARIA and Assistive Technologies Community Group (2026); PowerMapper Software (2025); Web Platform Tests (2026). ARIA-AT percentages are the share of required ("MUST") assertions passed with the most recent assistive technology version tested, except Minimal Data Grid, which reports all assertions. WCAG 2.2 removed success criterion 4.1.1 Parsing.

Other sources were reviewed and not used for current ratings. Steve Faulkner's HTML5 Accessibility tables are dated April 2022 (github.io edition) and August 2020 (html5accessibility.com) and include Internet Explorer 11. The a11ysupport.io results sampled for this paper date from 2022 and early 2023, and its maintainers note it may not be current. TetraLogical's 2025–2026 screen reader HTML support tables are current and useful as qualitative evidence, but they test mostly one browser per screen reader and give no scores. Appendix B assesses each source.

Author's assessment: The evidence supports a two-part conclusion. Engines now agree almost completely on what they compute, which is a significant achievement of Interop 2024. The remaining inconsistency lies mostly in the combination of browser and screen reader, in composite widgets, live regions, and media, and much of it is not yet covered by any consensus test. That is a coordination problem as much as an engineering one.

© 2026 Michael G. Paciello. All rights reserved.

6. Post-source accessibility

6.1 Framework

Post-source accessibility refers to changes made to content after it leaves its origin and before or during use. The W3C Accessibility at the Edge Community Group's Final Report distinguishes source-authorized changes, "controlled or approved by the website owner", from user-authorized changes, "added by the user, without website involvement or approval" (W3C Accessibility at the Edge Community Group, 2025). The author of this paper is a co-author of that report. Pages are composites: the median website uses 21 third parties on mobile and 23 on desktop (HTTP Archive, 2021), so no single author controls everything a user receives.

6.2 Browser-native post-source capabilities

Section 4 described the user-authorized capabilities browsers already ship. Two of them are often described inaccurately. User style sheets, frequently cited as a universal way for users to override author styles, are natively available only in Safari on macOS and, behind an advanced preference, in Firefox; Chromium removed them in 2014 (Chromium Project, 2013), so Chrome and Edge users need an extension. Caret browsing, sometimes described as a Firefox and Edge feature, is also available in Chrome.

6.3 Extensions

Extension APIs let third-party tools inject CSS, modify the DOM, and add ARIA attributes. Chromium extensions can inject user-origin CSS; the private `chrome.automation` tree API is not available to them (Section 3.1). Extensions include contrast and color tools, reading aids, text-to-speech tools, and overlay products that add accessibility widgets to pages. Their effectiveness varies, and the paper does not evaluate individual products.

6.4 Automatability

The Accessibility at the Edge Community Group's Final Report assesses how far each post-source capability can be automated, using four labels: "In many cases", "In some cases", "In no significant way", and "Does not apply" (W3C Accessibility at the Edge Community Group, 2025, section 1.7.3). The report defines "In many cases" as automation that can "successfully remediate and verify outcomes across a broad set of websites." These ratings are the group's expert judgment: the report does not describe a test corpus or measurements, and a W3C Community Group report is not a W3C Standard. Table 5 therefore sets the group's rating for selected capabilities beside independent published evidence and the author's own view.

Two patterns stand out. First, the evidence supports automated *detection* far more strongly than automated *repair*: in a 2017 test by the UK Government Digital Service, ten automated tools together found 71% of 143 known barriers on a test page, and 29% were found by none of them (Duran, 2017). Second, where content must be interpreted, as with images, form labels, headings, and speech, independent studies show useful but imperfect results that still need human review.

© 2026 Michael G. Paciello. All rights reserved.

Table 5. Automatability of selected post-source capabilities: Community Group assessment compared with independent evidence (W3C Accessibility at the Edge Community Group, 2025)

Capability	Community Group rating	Independent evidence	Author's assessment
Provide text alternatives for images	In many cases	Missing alt text is detected at scale (WebAIM, 2026). Generated descriptions improve with page context but still include plausible errors, which blind users tend to trust (MacLeod et al., 2017; Mohanbabu & Pavel, 2024)	Detecting missing alt text: in many cases. Producing correct descriptions: in some cases
Indicate language	In many cases	Language identification is highly accurate for long single-language text and less accurate for short or mixed-language text (Jauhiainen et al., 2019)	Agrees for the page's language; in some cases for changes of language within a page
Adjust text spacing, line height, and fonts to values the user chooses	In many cases	WCAG expects users to apply these through style sheets, bookmarklets, or extensions (W3C WAI, n.d.-c); no outcome studies found	Agrees
Improve hyperlink visibility	In many cases	No independent evaluation found	Agrees, with limited evidence
Associate form labels	In many cases	Missing labels are detected on 51% of home pages (WebAIM, 2026). Language-model repair reduces rule violations but does not fix every issue (Fathallah et al., 2025)	In some cases, because visual proximity does not always identify the intended label
Expose structural semantics (headings)	In many cases	No current independent evaluation of automated heading inference on the web found	In some cases, for headings inferred from visual styling alone
Provide captions for video	In many cases	W3C advises that automatic captions usually need significant editing (W3C WAI, 2024b). Error rates differ between speakers (Koenecke et al., 2020)	In some cases
Optimize color contrast	In some cases	Contrast failures are detected at scale (WebAIM, 2026); text over images and gradients is not reliably evaluated by automated rules (Deque Systems, n.d.)	Agrees
Manage focus through interactive processes	In some cases	No independent evaluation found	Agrees, with limited evidence
Handle dynamic pages accessibly (including live regions)	In some cases	No independent evaluation found; live-region support varies (Section 7.5)	Agrees, with limited evidence

© 2026 Michael G. Paciello. All rights reserved.

Capability	Community Group rating	Independent evidence	Author's assessment
Remediate ARIA roles and attributes	In some cases	No independent evaluation of automated ARIA repair found	Agrees, with limited evidence
Provide audio description	In no significant way	A research system produced short-clip descriptions that blind and low-vision participants preferred to descriptions written by novice humans (Li et al., 2025); timed, production-quality description has not been shown	In some cases in research settings; in no significant way in production
Rebuild or replace an interactive component	In no significant way	No independent evaluation found	Agrees

Evidence that a problem can be detected automatically is not evidence that an automated fix is correct for users. The studies cited measure different things (detection rates, rated description quality, and speech-recognition error rates) and are not directly comparable. One such system, AccessGuru, is reported by Fathallah et al. (2025) to have reduced an average accessibility-violation severity score by up to 84% on a 250-case syntax/layout evaluation sample drawn from a collection of roughly 3,500 real violations — a score reduction, not a count of violations fully fixed — while a separate 55-case semantic evaluation reported about 77% average similarity (Sentence-BERT cosine) to human-developer corrections; the authors stress that results still require human review.

7. ARIA as a post-source remediation strategy

7.1 Purpose and the first rule of ARIA

WAI-ARIA defines roles, states, and properties that convey semantics when native HTML cannot. WAI-ARIA 1.2 has been a W3C Recommendation since June 2023, and ARIA 1.3 is a Working Draft (W3C, 2023, 2026b). The widely cited "first rule of ARIA use" comes from the W3C document *Using ARIA*, not from the ARIA specification: "If you can use a native HTML element or attribute with the semantics and behavior you require already built in, instead of re-purposing an element and adding an ARIA role, state or property to make it accessible, then do so." W3C retired *Using ARIA* as a Discontinued Draft on 24 February 2026 and now points readers to the ARIA Authoring Practices Guide (APG) (W3C, 2026c).

7.2 How browsers implement ARIA

Browsers parse ARIA attributes, integrate them into the accessibility tree, apply the specifications' rules for invalid or prohibited usage, map them to platform APIs, and fire events when states change. They do not validate whether an author's use of ARIA is appropriate. Per Core-AAM 1.2, `role="button"` maps to ROLE_SYSTEM_PUSHBUTTON in MSAA and IA2, the Button control type in UIA, ROLE_PUSH_BUTTON in ATK/AT-SPI, and AXButton in the macOS AX API. A button with a defined `aria-pressed` value keeps ROLE_SYSTEM_PUSHBUTTON in MSAA and the Button control type in UIA, but maps to IA2_ROLE_TOGGLE_BUTTON in IA2, ROLE_TOGGLE_BUTTON in ATK/AT-SPI, and AXCheckBox with the AXToggle subrole in the AX API (W3C, 2026a).

© 2026 Michael G. Paciello. All rights reserved.

7.3 Effective and problematic post-source uses

Post-source ARIA can add missing accessible names, declare live regions, correct roles on elements that clearly function as widgets, and add descriptions. It cannot add behavior: `role="button"` on a `div` does not make it focusable or operable. Automated tools may infer purpose incorrectly, composite widget patterns need coordinated roles, states, and keyboard behavior that are hard to inject from outside, and keeping states synchronized with visual state requires knowledge of application logic.

7.4 What the data shows

In the 2026 WebAIM Million, 82.7% of home pages used ARIA (excluding landmark roles). Pages with ARIA averaged 59.1 detected errors, compared with 42 on pages without it (WebAIM, 2026); by the author's calculation, that is about 41% more. WebAIM cautions that this "does not necessarily mean that ARIA introduced these errors," because pages using ARIA also tend to be more complex. The association is consistent with the APG's guidance under the heading "No ARIA is better than Bad ARIA" (W3C WAI, n.d.-a).

> **Author's assessment:** For post-source remediation, this suggests that ARIA injection without a reliable understanding of content structure and purpose carries a risk of making matters worse.

7.5 Limitations and browser gaps

ARIA changes only what assistive technology is told. It does not alter appearance, keyboard behavior, or focus management; it offers little for users with cognitive or visual needs that are not met through a screen reader; and, as the APG cautions under "No ARIA is better than Bad ARIA", an ARIA role that is not backed by the matching behavior can leave users worse off than no ARIA (W3C WAI, n.d.-a).

Two gaps are well documented:

- **Live regions.** Microsoft's explainer for the `ariaNotify()` API states that ARIA live regions are inconsistently implemented (Microsoft Edge, n.d.). `ariaNotify()` shipped in Chrome 141 in September 2025, with partial support on some platforms, and in Firefox 150 in April 2026; Safari does not support it (Google, 2025; Mozilla, 2026a; MDN, 2026). ARIA-AT has no live-region test plan beyond alerts.
- **Complex composites.** There is no ARIA-AT test plan for tree or treegrid, and the APG treegrid pattern notes that "some implementations of treegrid may not provide focus to rows" (W3C WAI, n.d.-b).

7.6 Automation and AI

Machine learning, including AI models, can infer structure, describe images, and propose labels, but generated output can be plausible and wrong, and automated systems cannot confirm that a remediation works for assistive technology users. The Accessibility at the Edge report concludes that "while automation and AI can help, we should not expect these can replace humans entirely," and describes human remediation of automatically identified issues, with the content creator's participation, as "an effective and efficient approach to accomplish accessibility" (W3C Accessibility at the Edge Community Group, 2025, section 5.1). Independent evidence is consistent with the first part of that conclusion: automated tools miss a substantial share of known barriers (Duran, 2017), automatic

© 2026 Michael G. Paciello. All rights reserved.

captions usually need editing (W3C WAI, 2024b), and blind users tend to trust generated image descriptions even when they are wrong (MacLeod et al., 2017). No study was found that measures the efficiency of the combined approach the report recommends. Section 10 examines the on-device AI capability that engines have begun to ship, and the concerns raised about it.

8. Regulatory context

This section summarizes what current U.S. and European rules require of browsers. Appendix A gives the full comparison, including points that are frequently misstated. The description is factual and does not take positions on policy.

Section 508 (United States). The Revised 508 Standards (36 CFR Part 1194; compliance required from 18 January 2018) define "platform software" to include web browsers. Section 502.3 requires platform software to provide documented accessibility services, and states that applications that are also platforms "shall expose the underlying platform accessibility services or implement other documented accessibility services." Section 503.2 requires applications to honor platform settings for color, contrast, font, and focus cursor, but exempts applications designed to be isolated from their platform, including web applications. The standards incorporate WCAG 2.0 (U.S. Access Board, 2017), and a search of the Federal Register on 8 October 2026 found no Access Board proposed or final rule to update them since January 2024 (Federal Register, 2026). Section 508 binds federal agencies and reaches browser vendors through procurement. Google and Mozilla publish Accessibility Conformance Reports for Chrome and Firefox, and Apple's macOS report covers Safari features (Google, n.d.-f; Mozilla, 2024b; Apple, 2024).

EN 301 549 (Europe). EN 301 549 V3.2.1 (2021) is the harmonized standard cited under the Web Accessibility Directive for public sector websites and apps (European Commission, 2021a). It treats user agents as software (Clause 11), lists web browsers among platforms, and notes that "for web content, the underlying platform is the user agent." ETSI published V4.1.1, prepared by the CEN/CENELEC/ETSI Joint Working Group on eAccessibility, on 2 September 2026. It moves the web, document, and software baselines to WCAG 2.2 Level A and AA, adds user-preference clauses for web pages (9.7), documents (10.7), and software (11.7), moves authoring-tool requirements to clause 5.10, rewrites the real-time communication clause, and adds annexes mapping the standard to the European Accessibility Act (ETSI, 2026). As of 8 October 2026 it had not been cited in the Official Journal (European Commission, n.d.-a).

European Accessibility Act. The EAA (Directive (EU) 2019/882) has applied since 28 June 2025 to listed products, including consumer computers "and operating systems for those hardware systems", smartphones, and e-readers, and to listed services, including e-commerce, banking, and electronic communications. Standalone web browsers are not listed. Whether a browser bundled with an operating system falls within that operating system's scope is not settled by the Directive's text. No harmonized standard had been cited under the EAA as of 8 October 2026 (European Parliament and Council, 2019; European Commission, n.d.-a).

User preferences in the two frameworks. Section 508 exempts web applications from honoring platform user preferences (503.2). EN 301 549 V3.2.1 notes that for web content the underlying platform is the user agent, and V4.1.1 adds clauses titled "User preferences for web pages" (9.7) and "User preferences" for software (11.7). Neither framework requires browsers to provide personalized adaptation outside procurement.

© 2026 Michael G. Paciello. All rights reserved.

Author's assessment: Read together, these provisions are relevant to the browser-level personalization proposed here.

8.1 Liability and the user-controlled model

A question that arises whenever software changes what a user sees is whether the party making the change takes on legal responsibility for the result. For browser-level accessibility adaptation, the question has two directions: whether adapting a page shifts a site owner's legal obligations onto the browser vendor, and whether it relieves the site owner of those obligations. This section offers the author's assessment, not legal advice; the point is to identify the question, not to resolve it.

The obligations in the frameworks this paper examines attach to content and to the entities that provide it. Section 508 binds federal agencies and reaches vendors through procurement; the Web Accessibility Directive and, prospectively, the European Accessibility Act bind covered bodies and economic operators; and litigation under the Americans with Disabilities Act is brought against the covered entity whose service is at issue. In each case the duty runs to the provider of the content or service, not to the user's software. WCAG itself applies to web content, not to user agents (Section 1.5), and this paper has already noted that user-side aids such as automatically generated image descriptions do not alter the page's markup, so authors remain responsible for text alternatives under WCAG 1.1.1 (Section 4).

The user-authorized and source-authorized distinction from the Accessibility at the Edge framework (Section 6) is central here. A user-authorized adaptation is one the user invokes, on their own device, for their own use — the same category as a screen reader, browser zoom, forced-colors mode, or a reading view, all of which users already control today without transferring the site owner's obligations to the browser vendor. A browser that adapts a page at the user's request is exercising the user's long-recognized agency over their own experience, not republishing or warranting the site. On that reading, user-controlled browser adaptation does not move the site owner's legal duty onto the browser, and it is not designed to.

By the same logic, it does not relieve the site owner either. Because the adaptation happens on the user's device at the user's option, and may not occur at all for other users, the content as delivered is unchanged; the author's obligation to provide accessible content is not discharged by the possibility that some users' browsers will repair it. Browser-level personalization therefore complements author-side conformance rather than replacing it, and the legal pressure that currently drives content remediation is not removed by it.

Author's assessment: The liability question is best treated as a design constraint, not an obstacle. A capability that is user-initiated, user-controlled, and executed on the user's own device sits in the established category of assistive technology and user-agent configuration, where the user, not the software vendor, directs the adaptation and the content author's duties are unchanged. A source-authorized capability — one a site invokes, or one a vendor applies automatically to all users without their direction — raises different and harder questions, which is a further reason this paper favors the user-controlled model. These are the author's views; the allocation of liability is ultimately a legal question that merits analysis by counsel and by the standards and policy bodies named in Sections 8 and 12.

© 2026 Michael G. Paciello. All rights reserved.

9. Gap analysis

Because WCAG applies to content, this paper frames gaps not against WCAG conformance but against what users need from the user agent, and maps each to the UAAG 2.0 success criteria that already describe it.

Table 6. Gaps at the user agent layer

Gap	What exists today	What is missing	Related UAAG 2.0 criteria
1. Portable preference profile	Per-browser settings; a few sync within one vendor's account	A standard, portable description of a user's needs that any browser can apply, and per-site overrides	2.6.1, 2.6.3, 2.6.5
2. Standards-based adaptation of any page	Reading views; Firefox color override; forced colors on Windows; user style sheets in two browsers	Spacing, font, color, and layout adaptation applied to arbitrary pages from user preferences, without breaking functionality	1.4.1–1.4.6, 1.7.1–1.7.3, 1.8.13–1.8.15
3. Cognitive and reading support	Vendor-specific reading aids, dictionaries, translation, opt-in AI summaries	Consistent, preference-driven support across browsers; standard semantics such as WAI-Adapt	Partly: 1.9.1 (outline view), 2.7.1 (customize display of user interface controls)
4. Repair of missing semantics	Image descriptions in Chrome and Edge; PDF OCR in Chrome	Consistent, labeled, user-controlled repair for names, labels, headings, and landmarks, kept distinct from author-provided content	Guideline 1.2 (repair missing content), 1.10.1
5. Media alternatives	WebVTT captions everywhere; in-band audio tracks in Safari	`kind="descriptions"` in any engine; audio track APIs by default in Chromium and Gecko; browser-voiced, pausable descriptions	1.1.1, 1.1.4, 1.1.5, 1.1.6, 2.10.5
6. Consistent assistive technology output	Near-complete agreement on computed roles and names	Consensus test coverage for live regions, trees, comboboxes, and landmarks; platform-API tests	4.1.1, 4.1.2
7. Developer tooling	Accessibility trees, audits, and vision simulation in DevTools	A preview of what assistive technologies will announce	(not a UAAG criterion)
8. New engines and new platforms	Ladybird and Servo are building accessibility support	Accessibility requirements in Apple's alternative-engine criteria; funding for new engines	(not a UAAG criterion)

Several WCAG 2.2 Level AAA criteria help illustrate what browser support could add. Extended audio description (1.2.7) has no browser support. Visual presentation (1.4.8) is achievable by users only inside reading views or with user style sheets. Reading level (3.1.5) is approached only by opt-in AI summaries, which are not standards-based and whose accuracy is not guaranteed. Pronunciation (3.1.6) is

© 2026 Michael G. Paciello. All rights reserved.

supported only through user-initiated dictionary lookups, such as Edge's dictionary with audio; no browser exposes author-specified pronunciation.

10. On-device AI and the engines' stated concerns

The gaps in Section 9 describe work the browser could do. A recent change in browser capability bears directly on whether that work is now feasible: browser engines have begun shipping general-purpose, on-device AI that runs on the user's own machine. This section describes what has shipped, how a browser can act on a page, and the specific, documented concerns that engine and standards participants have raised about the general AI interface. The concerns are real engineering questions; this section sets each beside the narrower, user-controlled accessibility use to which this paper applies them. Nothing here depends on a particular vendor's product; the capability is the point.

10.1 What has shipped

Chrome exposes an on-device language model to code through its Prompt API. As of the Chrome for Developers documentation (last updated 26 August 2026), the Prompt API has been available to Chrome extensions since Chrome 138, and a version for web pages is rolling out in Chrome 148, where the documentation states it "is still being developed" (Google, n.d.-j). The model is Gemini Nano, run locally; the documentation gives no model size but states the feature needs at least 22 GB of free space on the profile's volume, and that the API is available to top-level windows and same-origin iframes but not to Web Workers (Google, n.d.-j). Microsoft's Edge, built on the same engine, has developed a comparable on-device capability; this paper does not independently verify Edge's current status.

> **Author's assessment:** The significance is not the specific product. It is that a mainstream engine now ships a local model that page and extension code can call, on consumer hardware, without sending page content to a server. Whatever one concludes about the general interface, the computational capability that a browser-level accessibility transformation would require is no longer hypothetical.

10.2 How a browser can act on a page

A browser already has complete access to the structures it builds from a page, and can modify them before or as the user receives them. Three layers are available, each described elsewhere in this paper:

- **The DOM.** The browser can read and modify the Document Object Model — adding a missing accessible name, correcting a role, inserting a heading. Extensions do this today through user-origin injection (Section 3.1). As Section 7.3 notes, injected ARIA changes only what assistive technology is told; it cannot add behaviour, so a repaired role on a non-interactive element does not make it operable.
- **The accessibility tree.** The browser computes the role, name, state, and relationship information it passes to assistive technologies (Section 2.1), and holds it in a cached, serializable copy in a privileged process (Section 2.3). This is the highest-leverage layer: it can repair what assistive technology receives without altering the visible page.
- **The style system.** User-origin styles can override author presentation — spacing, colour, contrast, font, reflow — the adaptation described in Gap 2 (Section 9).

© 2026 Michael G. Paciello. All rights reserved.

A browser-level accessibility transformation therefore combines a capability the browser has always had — reaching into these three layers — with a new one: a local model able to decide what to change. The reaching-in is not new; it is how extensions and assistive technologies already work. What is new is on-device inference capable of informing it. How good is “good enough” is task-dependent — inferring a missing label or simplifying a passage make different demands — and is bounded by the on-device hardware floor discussed in Section 10.1 and by the prototype’s preliminary, still-to-be-tested status noted in Section 10.3; the claim here is that the capability now exists and merits evaluation, not that any one model already meets a defined accessibility threshold across tasks.

> **Author's assessment:** This reframes the feasibility question. The paper's earlier sections show the browser already repairs malformed markup, reflows content, and translates pages. Acting on the accessibility tree or user-origin styles under the user's stated preferences is the same class of operation, now with a local model to guide it.

10.3 Who supplies and controls the intelligence

The post-source framework in Section 6 distinguishes source-authorized changes, controlled by the site owner, from user-authorized changes, made by the user without the site's involvement. The same distinction orders the ways browser-level AI could be applied, along a spectrum:

- **User-supplied intelligence.** The user runs a local model they control and directs it at any page. A working example is the “Smart Browser” prototype described by Dr. Gregg C. Vanderheiden — Professor Emeritus at the University of Maryland and founder and Director Emeritus of the Trace R&D Center, and presented at the M-Enabling Summit and to NDIA in 2026: built in roughly one person-month and running entirely on a five-year-old laptop, it already makes non-conforming pages satisfy a substantial share of WCAG success criteria through user-controlled settings. Vanderheiden characterizes these as preliminary, first-pass results that require further testing and acceptance by people with disabilities before such tools are relied upon; his work is supported in part by NSF Grant No. 2312370 and NIDILRR/ACL/HHS Grant No. 90REGE008. (Cited here with attribution, pending his confirmation; the figures and framing are his.) The user brings the model.
- **Vendor-supplied capability.** The engine provides a defined, standardized capability that applies accessibility transformations under the user's stated preferences. The engine supplies the model; the user still initiates and controls the adaptation.

Both act on the same three layers in Section 10.2, and both are user-authorized in the Section 6 sense: the user, not the site, decides. Presenting them as a spectrum rather than as competing proposals is deliberate. The case for browser-level accessibility does not depend on which end prevails; it is strengthened by holding both, because the user-controlled property — the safeguard that the concerns in Section 10.4 turn on — holds across the whole range.

10.4 The engines' documented concerns

When the general Prompt API was proposed for the web, engine and standards participants filed specific concerns in public review. These are reproduced here from the original issues, not from secondary summaries, because they bear directly on how a browser-level accessibility capability should be

© 2026 Michael G. Paciello. All rights reserved.

designed. They are documented concerns about a general interface, not settled positions for or against; where a position was proposed or later revised, this paper says so.

- Mozilla recorded a "negative" position on the Prompt API, stating that "the more general shape of this API has utility, but encourages model-specific behavior that increases interoperability risk," and that developers tuning to one model "would lead to worse behavior with other browsers or models (or sites blocking access)"; Mozilla added that the work "needs much more baking in userland before web APIs are shipped" (Mozilla, n.d.-i).
- In the WebKit standards-positions review, a reviewer proposed that WebKit oppose the API and raised that "the API has no user consent mechanism for running inference," that on-device inference can mean "the user pays dearly in battery life and device responsiveness, and sites benefit from free compute," that framing on-device inference "as inherently more private is also misleading," and that "model availability remains a fingerprinting vector." The opening review asked whether the interface "would work well with Apple Intelligence or other WebKit browsers" (WebKit, n.d.-a). The WebKit review subsequently resolved to a position of oppose on the Prompt API proposal, and the issue was closed as completed on 26 May 2026; that recorded position concerns this API proposal and not a future browser-level accessibility capability, which remains a separate question.
- The W3C Technical Architecture Group raised that "on-device models might not meet user, developer, and result expectations," that exposing only two sampling parameters creates "potential interop issues," and that licensing costs "could prevent some independent browsers from shipping a competitive implementation of this API." The TAG first recorded a lack of consensus, then rescinded that resolution and reopened the review, so no final TAG position stands (W3C TAG, n.d.-a).

Table R sets each documented concern beside the narrower accessibility use this paper proposes. The pattern is consistent: most concerns are about open-ended, site-invoked prompting of a general model. A user-controlled accessibility transformation — a constrained task, initiated by the user, benefiting the user — meets several of them by construction, and the rest become design requirements rather than objections.

Table R. Documented concerns about the general interface, and the user-controlled accessibility use 

Concern (and who raised it)	What it is about	How the accessibility use addresses it
Interoperability — model-specific prompting increases fragmentation risk (Mozilla; W3C TAG)	Open-ended prompts tuned to one model behaving differently on another	Accessibility transformations are a constrained, specifiable task — apply defined adaptations to defined preferences — not open-ended prompting. A defined capability surface with a shared test suite (one is in development, per the TAG thread) is far more standardizable than raw model access. The concern argues for a model-agnostic, specified interface, which is what this paper proposes.

© 2026 Michael G. Paciello. All rights reserved.

Concern (and who raised it)	What it is about	How the accessibility use addresses it
User consent for inference (WebKit review)	A site invoking inference without the user's knowledge or agreement	The accessibility use is user-initiated by definition: the user turns it on and states their preferences. Consent is intrinsic, not an added mechanism.
Resource and battery cost — "the user pays, the site benefits" (WebKit review)	A site invoking inference, consuming the user's battery and compute for the site's benefit	In the accessibility use the user is the direct and intended beneficiary, and the user initiates the work for their own access. The cost/benefit objection to sites invoking inference does not apply to a user invoking it for themselves.
On-device is not automatically more private (WebKit review)	Treating on-device as a privacy guarantee when model availability can still be observed	A fair caution. On-device processing does keep the user's page content on the device — an advantage over cloud remediation and third-party overlays — but model availability as a fingerprinting signal is a real design constraint to mitigate, not a reason against the approach. The paper states both.
Independent-browser viability — licensing could exclude smaller engines (W3C TAG)	Costly model licensing locking out independent browsers	Reinforces this paper's position (Sections 3.4–3.5, the roadmap, and the funding commitment in the challenge) that new and non-profit engines need accessibility work funded, and that the capability should rest on open, specifiable interfaces, not a single proprietary model.
Versionless or unstable model behaviour in production (raised generally for the general API)	A general model changing under applications that depend on it	Argues for a versioned, testable accessibility capability with defined behaviour and conformance tests — exactly the specified-interface approach this paper recommends, rather than exposing a raw model.
"Automated repair is unreliable; overlays failed" (anticipated; consistent with this paper's own caution in Sections 6–7)	Past automated-accessibility products that overpromised	The failures this paper documents were of vendor-side products sold as compliance. The approach here is user-side, user-controlled, and on-device, with a human in control and the automatability limits of Section 6 and Table 5 stated plainly. It is a different architecture, not a repeat — and this paper does not claim automated repair is complete or always correct.

Taken together, the documented concerns do not describe an insurmountable objection to browser-level accessibility. They describe the design of a general AI interface — and, read as requirements, they point toward the same thing this paper proposes: a defined, standardized, user-controlled capability rather than open-ended, site-invoked access to a proprietary model. A further concern, raised in review, is that model-driven adaptation is non-deterministic: the same page could be conveyed differently from one run to the next. This reinforces the case for a defined, versioned, testable capability with

© 2026 Michael G. Paciello. All rights reserved.

conformance tests over open-ended prompting, and for keeping the user in control of when and whether an adaptation is applied. Section 11 sets out the engineering roadmap; Section 12 shows the cost; Section 13 turns both into measurable commitments.

> **Author's assessment:** This is the same method the author has seen work in regulatory settings: take the other side's stated concerns seriously and on the record, and show that the specific proposal meets them. Here the concerns are not even settled positions; most are about a general interface, and the accessibility use answers them by being narrower, user-initiated, and user-controlled. What remains is not a technical barrier but a decision about whether to build it.

11. Roadmap for engine and browser teams

The roadmap separates work in shared engine code, which every browser on that engine inherits, from browser-product work, which each browser must build. Each phase lists measurable deliverables.

Table 7. Proposed 36-month roadmap

Phase and months	Engine work	Browser-product work	Measurable deliverables
1. Foundations (1–6)	Close remaining WAI-ARIA, AccName, HTML-AAM, and Core-AAM failures; expand platform-API tests; accessibility tree performance	Fix browser-UI accessibility defects	WPT pass rates for wai-aria, accname, html-aam, and core-aam published per release; ARIA-AT Candidate plans advanced to Recommended with browser teams
2. Preferences (7–12)	Expose user preferences to content through standard media queries and a preference model	Preference settings UI, profiles, sync, import and export	A shared preference vocabulary drafted with W3C (WAI-Adapt or successor); profile export and import between at least two browsers
3. Adaptation (13–24)	Preference-driven CSS overrides (spacing, fonts, color, reflow) that preserve functionality; `kind="descriptions"` and audio track APIs	Adaptation controls, per-site overrides, media controls	New WPT tests for preference-driven rendering; no regressions in WCAG 1.4.10 and 1.4.12 test pages
4. Cognitive and repair support (25–36)	Labeled repair of missing names, labels, and structure, kept distinguishable from author content	Reading support, simplified modes, comprehension aids	Published evaluation with users with cognitive and learning disabilities
Integration and quality assurance (1–36)	Cross-platform regression testing	Assistive technology compatibility testing	Results published against ARIA-AT and WPT

© 2026 Michael G. Paciello. All rights reserved.

EN 301 549 adds two smaller workstreams: reviewing authentication flows the browser controls, so that no single biological characteristic is the only means of identification (EN 301 549 clause 5.3), noting that in WebAuthn user verification is performed locally by the authenticator, through modalities such as a PIN, password, or biometric (W3C, 2026d). This workstream matters for accessibility as well as security: some users cannot reliably present a given biometric — for example, a user with no usable fingerprint, a blind user or one with nystagmus for whom an iris or face scan is unreliable, or a non-speaking user for whom voice verification is not an option — so an equally secure, fully accessible verification path must remain available to them. The second workstream supports real-time text in web communication, for which the IETF has specified T.140 text over WebRTC data channels (RFC 8865; Holmberg & Hellström, 2021).

12. The business case

Accessibility is usually argued for on ethical or legal grounds. This section makes the commercial case: what the investment costs, which of a browser vendor's own revenue drivers it protects or strengthens, and how small the gain needed to pay it back is. All inputs are sourced; projections are labeled as author's assessments.

11.1 The investment

Table 8. Cost model assumptions

Assumption	Value	Basis
Fully loaded cost per engineer-year, large US technology company	Low $300,000; base $400,000; high $500,000	Author's derivation: Levels.fyi (a compensation-data website) 2025 median total compensation of $312,000 for senior engineers and about $195,000 for all software engineers (second half of 2025), plus non-wage benefits from the Bureau of Labor Statistics Employer Costs for Employee Compensation (June 2026) and an author-assumed 15–30% overhead, gives about $350,000–$430,000 for a 60/40 senior blend and about $415,000–$505,000 for a senior-only team; the $300,000 low case allows for a less senior team (Levels.fyi, 2025; U.S. Bureau of Labor Statistics, 2026)
Fully loaded cost per engineer-year, non-profit or contractor	About $120,000–$150,000; $150,000 used	Servo's sponsorship tier of $10,000 per month to fund a developer (Servo Project, n.d.-a); Stack Overflow 2025 survey median for back-end developers in Germany, with author-assumed on-costs (Stack Overflow, 2025)
Mozilla cost per engineer-year	$250,000 (placeholder)	No public compensation data; author estimate
Phase durations and staffing	Durations as in Table 7; staffing as in Table 9	Author estimate
Share of each workstream in shared engine code	30% to 80% by workstream	Author estimate, offered for correction by engine and browser teams

© 2026 Michael G. Paciello. All rights reserved.

Table 9. Effort and cost by workstream (low, base, and high scenarios combine low, base, and high staffing with low, base, and high rates)

Workstream	Engineer-months (low / base / high)	Low	Base	High
Phase 1: API mapping and ARIA completeness	48 / 60 / 72	$1.20M	$2.00M	$3.00M
Phase 2: Preference framework	60 / 75 / 90	$1.50M	$2.50M	$3.75M
Phase 3: Content adaptation	180 / 240 / 300	$4.50M	$8.00M	$12.50M
Phase 4: Cognitive and repair support	180 / 240 / 300	$4.50M	$8.00M	$12.50M
Integration and quality assurance (36 months)	180 / 270 / 360	$4.50M	$9.00M	$15.00M
EN 301 549: authentication flow review	15 / 22.5 / 30	$0.38M	$0.75M	$1.25M
EN 301 549: real-time text support	30 / 45 / 60	$0.75M	$1.50M	$2.50M
Total per engine-and-browser pair	693 / 952.5 / 1,212	$17.3M	$31.8M	$50.5M

Spread over 36 months, that is about $5.8 million (low), $10.6 million (base), or $16.8 million (high) a year, funded as recurring engineering work rather than as a one-time capital project. Peak concurrent staffing is 20, 27.5, and 35 engineers. At non-profit rates the same plan costs about $8.7 million to $15.2 million in total.

Shared engine work lowers the cost per browser. Under the engine-share assumptions in Table 8, about half the base-case cost ($15.6 million) is engine work and half ($16.2 million) is browser-product work. Work landed once in Chromium reaches Chrome, Edge, Samsung Internet, and Opera, browsers that together accounted for 76.5% of page views in September 2026 (StatCounter, 2026; a figure that includes Chrome on iOS, which uses WebKit outside the EU and Japan; Apple, n.d.-a, n.d.-b). For four browsers on one engine, building separately would cost about $127 million in the base case; sharing the engine work and building four product layers would cost about $80 million. Chromium already shows shared stewardship of this kind: three of the six owners listed for its shared `ui/accessibility` directory use microsoft.com addresses (Chromium Project, 2026b).

11.2 Three revenue pools

Browsers are free to users, so any return would arrive through the revenue a browser drives. User-agent accessibility touches three revenue pools that together run to hundreds of billions of dollars. Figures below are marked as **reported** (company filings or official statements) or **forecast** (third-party projections, with publisher and date).

Pool 1: AI agents and agentic commerce

Size (forecast). Forecasts of U.S. consumer spending that AI agents will initiate, influence, or complete by 2030 range from $190 billion to $385 billion (Morgan Stanley, 2025), through $300 billion to $500 billion (Bain & Company, 2025), to "$900 billion to $1 trillion" of "orchestrated revenue" in U.S. retail goods, and $3 trillion to $5 trillion globally (McKinsey & Company, 2025). These are transaction volumes,

© 2026 Michael G. Paciello. All rights reserved.

not revenue that accrues to browser vendors, and they rest on adoption assumptions stated by each forecaster.

Monetization (reported). Vendors already charge for browser agents. Chrome's auto browse is available to Google AI Pro and Ultra subscribers in the United States and India (Google, n.d.-i). Microsoft retired Copilot Mode in Edge in May 2026 and made its agent feature, "Browse with Copilot," part of Microsoft 365 Premium for subscribers in the United States (Microsoft, 2026d), a plan announced at $19.99 a month (Microsoft, 2025b). Opera Neon is sold at $19.90 a month (Opera, 2025). The audiences are large: the Gemini app passed 1 billion monthly users (Google, 2026c); ChatGPT reported more than 900 million weekly active users and more than 50 million consumer subscribers (OpenAI, 2026); and Microsoft 365 Copilot passed 30 million paid seats (Microsoft Corporation, 2026a).

Reliability is the constraint (reported). In a study of 300 tasks on 136 live websites, human evaluators found that most agents succeeded about 30% of the time and the best 61.3% (Xue et al., 2025; preprint). On the WebArena benchmark, by comparison, humans succeeded 78.24% of the time and the best agent in the original paper 14.41% (Zhou et al., 2023).

The link to accessibility (reported). The Chrome team states that "a well-formed accessibility tree is the primary way AI agents understand your page" (Kulikowski & Hablich, 2026), and Lighthouse now includes an "Agentic Browsing" category that checks, among other things, that interactive elements have programmatic names (Kulikowski & Hablich, 2026). Google's guidance for agent-friendly websites calls the accessibility tree "a high-fidelity map" for agents (Kulikowski & More, 2026). Microsoft's Playwright MCP server operates the browser through "structured accessibility snapshots" (Microsoft, n.d.-c), and Chrome DevTools MCP's page snapshot is "based on the a11y tree" (Google, n.d.-g). OpenAI told publishers that its Atlas browser agent used "ARIA tags—the same labels and roles that support screen readers" (OpenAI, n.d.-a); OpenAI has since deprecated Atlas, which was scheduled to stop working on 9 August 2026, and is moving browser-based agent capabilities into ChatGPT and Codex (OpenAI, n.d.-b). The WebArena research benchmark supports several page representations, and its baseline agents use the accessibility tree (Zhou et al., 2023). An industry study by Merj, a web research company, swapped the ARIA labels of a Save and a Cancel button: one agent clicked Cancel first in 25 of 25 runs, while two others went to Save. Averaged across agents, success on an icon-only button with no accessible name was about 34% in November 2025 and about 45% in March 2026 (Zecchini, 2026). That study is not peer-reviewed and its dataset is unpublished, and the study calls the accessibility-tree explanation "the most plausible mechanism rather than an established one." Not every agent relies on the accessibility tree, and no peer-reviewed study yet isolates accessibility quality as a cause of agent success.

> **Author's assessment:** Agent reliability on real websites is what stands between vendors' agent subscriptions and the forecast transaction volumes, and the accessibility tree is the input vendors themselves say agents rely on. Engine work that makes the tree complete, correct, and fast is therefore agent-product work. One percent of the lowest 2030 forecast is about $1.9 billion of transactions a year.

Pool 2: Search, advertising, and the longevity economy

Vendor revenue (reported). Alphabet earned more than 70% of its $402.8 billion of 2025 revenue from online advertising (Alphabet Inc., 2026), that is, more than $280 billion (author's calculation). Apple's Services net sales were $109.2 billion in fiscal 2025, at a 75.4% gross margin, and include licensing

© 2026 Michael G. Paciello. All rights reserved.

revenue from Google whose amount Apple does not disclose (Apple Inc., 2025). For Apple, the court record in *United States v. Google LLC* reports Google payments of about $20 billion in 2022 (U.S. Department of Justice, 2026b). Mozilla's 2024 royalties were $498.2 million (Mozilla Foundation, 2025).

The users (reported and forecast). Americans aged 50 and over contributed $8.3 trillion to the U.S. economy in 2018, and "fifty-six cents of every dollar spent in the United States in 2018 came from someone 50 or older" (Terrell, 2019). AARP's global study with Economist Impact estimated that people aged 50 and over accounted for $35 trillion of consumer spending in 2020 and forecast $96 trillion by 2050 (AARP, 2022). Working-age U.S. adults with disabilities had $674.3 billion of disposable income in 2024 (American Institutes for Research, 2026). About 830 million people worldwide were aged 65 or over in 2024, 10.2% of the population (World Bank, 2026); in the United States the share was about 18.9% on 1 July 2025 (U.S. Census Bureau, 2026, Table 1; author's calculation), and in the European Union 22.0% (Eurostat, 2026a). Disability rises with age: in the United States, 24.3% of people aged 65 to 74 and 45.9% of those aged 75 and over report a disability (U.S. Census Bureau, 2024). WHO estimates that at least 2.2 billion people have a near or distance vision impairment (World Health Organization, 2026), and that hearing loss affects more than 1.5 billion people (World Health Organization, 2021).

The gap (reported). Most older adults are online, but fewer transact: 90% of U.S. adults aged 65 and over used the internet in 2025 (Pew Research Center, n.d.), yet in the European Union in 2025, 55% of internet users aged 65 to 74 bought goods or services online, against 78% of all internet users (Eurostat, 2026b). In 2021, 15% of U.S. adults with a disability said they never go online, against 5% of those without one (Pew Research Center, 2021).

> **Author's assessment:** No published source measures how much search, advertising, or commerce revenue comes from older users or users with disabilities. What the figures do show is scale: the users whose experience browser accessibility most affects are a large and growing share of the population every browser serves, and of its spending, and at Alphabet's advertising base one-tenth of one percent is more than $280 million a year.

Pool 3: Per-seat enterprise and public-sector revenue

The revenue (reported). Microsoft reported more than 450 million paid Microsoft 365 commercial seats (Microsoft Corporation, 2026b). Its Productivity and Business Processes segment had fiscal 2026 revenue of $140.0 billion and operating income of $83.9 billion, a 59.9% operating margin (Microsoft Corporation, 2026a; author's calculation). Edge for Business is included with Microsoft 365 plans (Microsoft, n.d.-d). Chrome Enterprise Core is offered at no cost, and Chrome Enterprise Premium is listed at $6 per user per month (Google, n.d.-h).

Accessibility is a condition of sale (reported). U.S. federal acquisitions of ICT "shall meet the applicable ICT accessibility standards" unless an exception or exemption applies (Federal Acquisition Regulation, n.d.), and federal guidance tells vendors that "it is not voluntary to complete an ACR if you wish the government to consider purchasing your product" (U.S. General Services Administration, n.d.-b). Google publishes Accessibility Conformance Reports for Chrome, ChromeOS, and Workspace for that purpose (Google, n.d.-f). In the European Union, Directive 2014/24/EU requires that technical specifications for procurement intended for use by natural persons take into account accessibility criteria for persons with disabilities or design for all users, except in duly justified cases (European Parliament and Council, 2014, Article 42). EU public authorities spend about €2.6 trillion a year, about 15% of EU GDP (European Commission, n.d.-b), and the U.S. budget for civilian federal IT was about $67.9 billion in FY2026

© 2026 Michael G. Paciello. All rights reserved.

(Bracken & Mitchell, 2026). Under the Department of Justice's ADA Title II rule, state and local government entities, including public schools and public colleges and universities, must make web content and mobile apps conform to WCAG 2.1 Level AA, subject to stated exceptions, by 26 April 2027 for entities with a total population of 50,000 or more, or 26 April 2028 for smaller entities and special district governments (U.S. Department of Justice, 2026a).

Precedent (reported). In 2010, the Departments of Justice and Education wrote jointly to college and university presidents, after settlements over a pilot of an inaccessible e-book reader: "It is unacceptable for universities to use emerging technology without insisting that this technology be accessible to all students" (U.S. Department of Justice & U.S. Department of Education, 2010).

Enforcement is uneven (reported). In GSA's FY2025 government-wide assessment, "inclusion of ICT accessibility in acquisition processes is high but enforcement is low" (U.S. General Services Administration, 2026). In its FY2024 assessment, 50% of reporting entities said they regularly, frequently, or almost always obtain Accessibility Conformance Reports for all products they buy (U.S. General Services Administration, 2024).

> **Author's assessment:** Accessibility does not create this per-seat revenue, but in public-sector and education markets it is a condition of keeping it, and the ADA Title II dates bring state and local governments, public schools, and public universities into scope in 2027 and 2028. At Microsoft's Productivity and Business Processes scale, one-tenth of one percent of revenue is $140 million, or about $84 million of operating income at the segment's average margin.

A related market signal. U.S. federal procurement guidance contemplates testing by an independent third party as one way to assess products (U.S. General Services Administration, n.d.-a), and a market for accessibility testing and remediation services exists. This paper does not size that market or rely on it.

> **Author's assessment:** Third-party accessibility spending suggests that buyers value accessibility outcomes, and enterprise browser offerings are one general avenue through which browser vendors could serve that demand. (The author's employer provides digital accessibility products and services; see Disclosures.)

11.3 What a small change is worth

Table 10. Value of a small change in each revenue pool, compared with the base-case annual cost of about $10.6 million (author's calculations; sensitivities, not forecasts)

Revenue pool	Base	0.1% of base	1% of base	Times the annual cost, at 0.1%
Alphabet online advertising (reported, 2025)	More than $280 billion	More than $280 million	More than $2.8 billion	More than 26
U.S. agentic commerce in 2030 (lowest forecast, Morgan Stanley)	$190 billion of transactions	$190 million	$1.9 billion	About 18

© 2026 Michael G. Paciello. All rights reserved.

Revenue pool	Base	0.1% of base	1% of base	Times the annual cost, at 0.1%
Microsoft Productivity and Business Processes (reported, FY2026)	$140.0 billion	$140 million	$1.4 billion	About 13
Apple Services (reported, FY2025)	$109.2 billion	$109 million	$1.1 billion	About 10
Mozilla royalties (reported, 2024)	$498.2 million	$0.5 million	$5.0 million	About 0.05 (pays back at about 2.1%)

These are sensitivities, not projections: the paper does not claim that accessibility work would move any of these pools by a given amount, and the agentic commerce row is transaction volume, not vendor revenue. They show scale. The program costs about $10.6 million a year; a change of one-tenth of one percent in any of the three reported revenue lines would equal about 10 to more than 26 times that cost.

Margin. The program is a fixed operating cost. Any revenue it protects or adds would land in high-margin lines: Microsoft's Productivity and Business Processes segment earned a 59.9% operating margin in fiscal 2026, and Apple's Services a 75.4% gross margin in fiscal 2025 (Microsoft Corporation, 2026a; Apple Inc., 2025; author's calculation for Microsoft). These are segment averages, not the margin on any specific revenue, and they are operating and gross margins rather than EBITDA.

> **Author's assessment:** For the largest vendors, this is the shape of an attractive investment: a small fixed cost, shared across products through the engine, set against revenue pools in the hundreds of billions that the work protects or enlarges. The decision does not turn on whether the work pays back; it turns on whether a vendor's agents, search, and enterprise products are reliable for the users and buyers who depend on accessibility before a competitor's are.

11.4 Breakeven

Table 11. What it takes to pay back the base-case annual cost of about $10.6 million (author's calculations from the sources in Section 12.2)

Measure	Result	Basis
Alphabet, Apple, Microsoft: cost in time of revenue	About 13 to 17 minutes of one year's revenue	Alphabet $402.8 billion (2025), Apple $416.2 billion (FY2025), Microsoft $331.8 billion (FY2026)
Mozilla: cost in time of revenue	About 5.7 days of 2024 revenue	$680.4 million total revenue and support
Mozilla: Firefox clients retained or won	About 4.4 million (about 2.1% of monthly active clients); 2.4 million to 6.9 million across scenarios	$2.43 royalties per client per year (Mozilla, n.d.-h)

© 2026 Michael G. Paciello. All rights reserved.

Measure	Result	Basis
Apple: devices retained or won	At least about 1.1 million (under 0.1% of the active base); 0.6 million to 1.7 million across scenarios	At most about $10 per device per year (Apple, 2023)
New subscription revenue	About 44,000 subscribers to a $19.99-a-month plan	$239.88 per subscriber per year
New enterprise seats	About 147,000 Chrome Enterprise Premium seats at list price	$72 per user per year

Per-client and per-device values divide reported revenue by reported user counts. Mozilla's royalties of $498.2 million divided by Firefox's 2024 average of 205.3 million monthly active clients come to about $2.43 per client per year; Mozilla's Public Data Report describes this measure as desktop and mobile clients active in the past 28 days, though the data file is published in its desktop section, and clients are installations, not people. Google's payments to Apple of about $20 billion in 2022, against the "more than 2 billion" active devices Apple reported in February 2023, come to at most about $10 per device. No sourced user counts were found for Chrome or Edge. For Mozilla, retaining or winning 0.5% of its 210.1 million monthly active clients (week of 28 September 2026) would be worth about $2.5 million to $2.6 million a year in royalties at the 2024 average, 1% about $5.1 million, 2% about $10.2 million, and 5% about $25.5 million. These are averages, and the average value of a client is not the value of the specific clients that accessibility work would retain.

11.5 What it means for each vendor

Table 12. Base-case annual cost compared with vendor resources

Organization (fiscal year)	Revenue	Research and development or software development expense	Base-case annual cost as share of revenue
Alphabet (FY2025)	$402.8 billion	$61.1 billion	0.003%
Apple (FY2025)	$416.2 billion	$34.6 billion	0.003%
Microsoft (FY2026)	$331.8 billion	$35.6 billion	0.003%
Mozilla Foundation and subsidiaries (2024)	$680.4 million	$290.4 million (software development)	1.6% at large-company rates; about 1.0% at the author's $250,000 placeholder rate
Servo	About $7,000–$8,000 per month in recurring donations (February–July 2026), plus a €545,400 grant for 2025–2026	Not applicable	Not comparable: the non-profit-rate base case is about $4 million per year
Ladybird	Budget not published	Not applicable	Not comparable

© 2026 Michael G. Paciello. All rights reserved.

Sources: Alphabet Inc. (2026); Apple Inc. (2025); Microsoft Corporation (2026a); Mozilla Foundation (2025); Servo Project (2026a, 2026b); Sovereign Tech Agency (n.d.); Ladybird Browser Initiative (n.d.).

For Mozilla, the profit-and-loss view adds context to the revenue share. Its 2024 total revenue and support exceeded expenses by $92.2 million before tax and $72.1 million after tax, but that total included about $95.4 million of investment income; excluding investment income and gains, revenue was slightly below expenses (Mozilla Foundation, 2025; author's calculation). About 86% of Mozilla's 2024 revenue from customer contracts came from a single customer, which its audited statements do not name. In *United States v. Google LLC*, the court's remedies decision allows Google to continue paying browser developers for default search placement on a non-exclusive basis, with agreements limited to one year (United States v. Google LLC, 2025). Both parties have appealed to the D.C. Circuit: Google challenges the ruling, and the plaintiffs ask the court to revisit the decision not to ban such payments. Mozilla filed an amicus brief supporting neither party (Courthouse News Service, 2026; MediaPost, 2026).

Author's assessment: The outcome may affect Mozilla's revenue and therefore its capacity to fund work of this kind.

Author's assessment: For the three largest vendors, cost is not the constraint; prioritization and cross-vendor coordination are. For Mozilla the roadmap is a material commitment best shared through upstream work. For Servo and Ladybird it would require dedicated external funding. Shared work, through upstream engine code, WPT, ARIA-AT, and W3C specifications, is the most efficient way to spread the cost.

13. The challenge to the browser industry

The business case leads to a specific challenge, addressed to the browser industry as a whole rather than to any one company: ten commitments, each measurable from public data, so anyone can check whether it has been met.

Author's assessment: Most of these commitments are also product milestones: for AI agents that work reliably on real websites, and for enterprise products that pass procurement review.

Table 13. Ten measurable commitments

Commitment	How it is measured	By
1. Publish accessibility test results with every stable release	WPT accname, wai-aria, html-aam, and core-aam pass rates published per release for every major engine	30 June 2027
2. Make accessibility a scored Interop focus area again, including platform-API tests	Accessibility focus area in the Interop 2028 selection, with stable-release scores of at least 95%	Interop 2028 selection
3. Close the assistive technology test gap	At least four new W3C ARIA-AT test plans at Recommended, covering live regions, combobox, landmarks, and tree or treegrid	31 December 2027

© 2026 Michael G. Paciello. All rights reserved.

Commitment	How it is measured	By
4. Reach consistent screen reader output on the core widgets	At least 95% of required ARIA-AT assertions passed by every tested screen reader and browser combination on all Recommended plans	31 December 2028
5. Ship text audio descriptions	`<track kind="descriptions">` and audio track APIs enabled by default in at least two engines	31 December 2027
6. Agree on a portable accessibility preference format	A W3C draft specification, implemented for import and export in at least two browsers from different vendors	Draft by 31 December 2027; implementations by 30 June 2028
7. Adapt any page to the user's stated needs	Preference-driven spacing, font, color, and reflow adaptation in at least two engines, with new WPT tests	31 December 2028
8. Preview what assistive technology will announce	A screen reader output preview in the developer tools of at least two browsers	31 December 2027
9. Make AI-agent readiness and accessibility one metric	Scored (not informational-only) developer-tool audits of accessibility and agent readiness built on the same accessibility-tree checks, documented publicly, in at least two browsers	30 June 2027
10. Fund the open engines	Pooled funding sufficient for the non-profit-rate base case (about $4 million a year per engine) committed to Servo and Ladybird accessibility work	31 December 2027

Author's assessment: None of these commitments requires new technology. Each requires a decision to fund work whose payback threshold, as Section 12 shows, is small relative to the revenue or user base of each of the four largest vendors, and the public record will show which vendors made that decision.

For W3C and other standards bodies

1. Decide where user agent accessibility requirements will live now that UAAG 2.0 will not advance, and connect them to WAI-Adapt and WCAG 3 work.
2. Resume WAI-Adapt publication, or charter a successor focused on a user-preference vocabulary that browsers can implement.
3. Continue Interop accessibility investigations toward a scored platform-API focus area.

Options for policymakers (stated neutrally)

1. Updating the Section 508 Standards' WCAG 2.0 baseline to WCAG 2.2 would align the U.S. federal web content baseline with EN 301 549 V4.1.1.
2. Clarifying whether browsers bundled with operating systems fall within the European Accessibility Act's product scope would reduce uncertainty for vendors.

© 2026 Michael G. Paciello. All rights reserved.

3. Including accessibility conformance criteria alongside the web-platform criteria already required of alternative browser engines on mobile platforms could help ensure that new engines address accessibility from the start.

14. Questions for discussion

1. Which engine layer, the cached accessibility tree, the style system, or a new component, is the right place for preference-driven adaptation?
2. What would a minimal, portable preference format need to contain, and who should own it?
3. Which ARIA-AT gaps would most improve consistency if closed in 2027?
4. Should user-side repairs be exposed to assistive technologies differently from author content?
5. Are the engine-share estimates in Table 8 reasonable? Engine teams are invited to correct them.
6. Which of the ten commitments in Table 13 can be met first, and which need a change in date or measure to be realistic?
7. Can vendors publish data that would test the business case directly, such as retention among users who turn on accessibility features, or agent task success on pages with complete accessibility trees?
8. Where does legal responsibility sit when a user's browser adapts a page for accessibility? If the adaptation is user-initiated and runs on the user's device, does it leave the content author's obligations unchanged — as a screen reader or reading view does today — and what, if anything, would change that analysis for a vendor-provided or automatically applied capability?

15. Conclusions

Browser engines now share a common foundation: converged architectures, near-complete agreement on computed semantics, and a growing set of user-side aids. What they have not built is a standard way for a person to state their needs once and have every page adapted to them.

The case for this work can be made in commercial terms. Section 12 shows that the base-case cost is minutes of annual revenue for the three largest vendors, that the work touches revenue pools worth hundreds of billions of dollars (AI agents, search and advertising from an aging population, and per-seat enterprise and public-sector sales), and that for Mozilla it pays back if the work retains or wins about 2.1% of Firefox's monthly active clients. Section 13 turns that case into ten dated commitments that anyone can check.

> **Author's assessment:** The obstacles are not mainly technical. They are the absence of a shared preference model, of consensus tests for assistive technology output, of a maintained user agent guideline, and, for the smaller engines, of funding. Each of those is something browser and engine teams, standards groups, accessibility specialists, and funders can act on together.

© 2026 Michael G. Paciello. All rights reserved.

Appendix A. U.S. and European regulatory comparison

Table A1 compares Section 508 and EN 301 549 on the topics most relevant to browsers. Table A2 lists points about these frameworks that are frequently misstated in discussions of browser accessibility.

Table A1. Section 508 and EN 301 549 compared on browser-relevant topics

Topic	Revised Section 508 Standards	EN 301 549 V3.2.1 (cited under the Web Accessibility Directive)	EN 301 549 V4.1.1 (published 2 September 2026, not yet cited)
Web content baseline	WCAG 2.0 Level A and AA (E205.4)	WCAG 2.1 Level A and AA (Clause 9)	WCAG 2.2 Level A and AA; 4.1.1 Parsing void
Software, including browsers	WCAG 2.0 applied to software UI (E207.2); platform software defined to include web browsers	Clause 11 applies to software; user agents named; web browsers listed as platforms	Clause 11 retitled "Non-web software"
Interoperability with assistive technology	502	11.5	11.5
User preferences	503.2; web applications exempt	11.7; "for web content, the underlying platform is the user agent"	9.7 (web pages), 10.7 (documents), 11.7 (software)
Biometrics	403.1: not the only means; two dissimilar biometrics permitted	5.3: not reliant on one biological characteristic	5.3 (title unchanged)
Preservation of accessibility information	404.1 (transmission or conversion)	5.4 (conversion)	5.4
Privacy	405.1 (hardware)	4.2.11 functional performance statement	4.2.11 (renamed functional performance criterion)
Authoring tools	504, including PDF/UA-1 export	11.8 (repair suggestions only where the tool checks)	5.10
Real-time text	412.5 reserved; 412.8 legacy TTY support	6.2, including transmission within 500 ms	Revised 6.2; 6.7 Total conversation; references include RFC 8831
Video communication	412.7 (quality sufficient for sign language)	6.5 (resolution, frame rate, synchronization)	6.5
Cognitive accessibility	302.9	4.2.10; informative Annex D	4.2.10; Annex D; new WCAG 2.2 criteria

© 2026 Michael G. Paciello. All rights reserved.

Topic	Revised Section 508 Standards	EN 301 549 V3.2.1 (cited under the Web Accessibility Directive)	EN 301 549 V4.1.1 (published 2 September 2026, not yet cited)
Legal reach	Federal agencies' ICT; vendors through procurement; complaints and civil actions under 29 U.S.C. 794d(f)	Mandatory for public sector bodies under the WAD; procurement; contracts	Would give a presumption of conformity under the EAA, to the extent cited in the Official Journal

Sources: U.S. Access Board (2017); ETSI (2021, 2026); European Commission (2021a); 29 U.S.C. § 794d (Legal Information Institute, n.d.).

Table A2. Frequently misstated points about U.S. and European requirements

Common statement	What the sources say
EN 301 549 V3.2.1 is the standard organizations must meet to demonstrate EAA compliance	No harmonized standard had been cited under the EAA as of 8 October 2026. V3.2.1 is cited under the Web Accessibility Directive. Harmonized standards are voluntary routes to a presumption of conformity.
V4.1.1 is forthcoming	Published 2 September 2026; Official Journal citation pending
Section 508 does not address biometrics, authoring tools, or privacy	It does: 403.1, 504, and 405.1
Preservation of accessibility information is unique to Europe	Section 508 404.1 covers it
Authoring tools must provide accessibility checking (11.8)	11.8.4 requires repair suggestions only where the tool already checks; authoring tools are clause 5.10 in V4.1.1
Browser vendors must meet EN 301 549	Standalone browsers are not listed in the EAA; vendors meet EN 301 549 mainly through procurement and contracts
The EAA explicitly provides consumer rights of action	Article 29 requires Member States to provide consumer actions under national law
The EAA requires accessibility statements	The EAA requires service information in terms and conditions or an equivalent document (Annex V); accessibility statements are a Web Accessibility Directive requirement. U.S. federal websites are directed to publish accessibility statements by OMB Memorandum M-24-08 (Office of Management and Budget, 2023).
EN 301 549 says 87 million people in Europe have disabilities or temporary impairments	The European Commission estimates about 87 million people in the EU have some form of disability (European Commission, 2021b)
Browsers should support RFC 4103 for WebRTC real-time text	RFC 4103 is an RTP payload format; the IETF specification for real-time text over WebRTC data channels is RFC 8865

© 2026 Michael G. Paciello. All rights reserved.

Other jurisdictions. Canada published CAN/ASC-EN 301 549:2024, an identical adoption of EN 301 549 V3.2.1, for federally regulated sectors (Accessibility Standards Canada, 2024). AS EN 301 549:2020 is used in Australian government ICT procurement guidance (for example, NSW Government, 2023).

U.S. state and local government web content. The Department of Justice's 2024 rule under Title II of the Americans with Disabilities Act adopts WCAG 2.1 Level AA for state and local government web content and mobile apps. An interim final rule of 20 April 2026 moved the compliance dates to 26 April 2027 for public entities with a total population of 50,000 or more, and to 26 April 2028 for smaller entities and special district governments, without changing the substantive requirements (U.S. Department of Justice, 2026a). The rule regulates content, not user agents.

Appendix B. Evidence sources

Table B1. Sources assessed for browser support evidence

Source	Maintainer	Latest data	What it measures	Used in this paper
Interop 2024 dashboard (wpt.fyi)	Web Platform Tests project and browser vendors	Final 2024 results; stable data through February 2025	Computed roles and accessible names	Yes
Web Platform Tests accessibility directories	Web Platform Tests project	Daily runs; 6 October 2026 cited	Computed roles and names; some platform mappings	Yes
W3C ARIA-AT	W3C ARIA and Assistive Technologies Community Group	Continuous; read 8 October 2026	Screen reader output for APG examples	Yes, marked provisional for Candidate plans
PowerMapper screen reader tests	PowerMapper Software	14 December 2025	Screen reader and browser reliability on fixed test cases	Yes, individual figures cited
MDN browser-compat-data and caniuse	Mozilla and community; Alexis Deveria and community	October 2026	Feature presence by browser version	Yes
TetraLogical screen reader HTML support tables	TetraLogical (Steve Faulkner, ed.)	Blog July 2025 (updated Aug 2026); lookup updated May 2026	Qualitative screen reader behavior for HTML elements	Qualitative only
HTML5 Accessibility (Steve Faulkner)	Steve Faulkner	April 2022 and August 2020	HTML feature accessibility support	No: out of date
a11ysupport.io	Michael Fairchild and community	Sampled results 2022–2023	Assistive technology support for HTML and ARIA	No: out of date
Accessibility Compatibility	lola's lab and Scaled Accessibility	Early development	Accessibility-support across browser and	Noted as an emerging

© 2026 Michael G. Paciello. All rights reserved.

Source	Maintainer	Latest data	What it measures	Used in this paper
Dataset (ACD)	(with Mozilla, WebDX CG, Baseline, ARIA-AT)	(2026)	assistive-technology combinations, for developer tools	source; not yet a figure source (early development)

Licenses: caniuse data is CC BY 4.0; MDN browser-compat-data is CC0; ARIA-AT is under the W3C Document License; TetraLogical's tables and a11ysupport.io are GPL-3.0; PowerMapper's results are copyrighted, so this paper cites individual figures only.

Disclosures

This paper was prepared with the assistance of AI (Anthropic's Claude), which was used to locate and verify sources and to draft text. The author reviewed every statement and source.

The author writes in a personal capacity. He is currently employed by AudioEye, Inc., which sells digital accessibility products and services, including AI-agent-based accessibility auditing and developer tools announced in October 2026. The views in this paper are his own and do not represent his employer. This paper uses no AudioEye data, products, or materials, and AudioEye did not fund it and has not reviewed it. The author is a co-author of the W3C Accessibility at the Edge Community Group Final Report cited in Sections 6 and 7, and of Laux et al. (1996), cited in Section 1.1.

About the author

Michael G. Paciello has worked in digital accessibility for more than 40 years. He began his career at Digital Equipment Corporation (DEC) in 1982, where he was a Senior Usability and Accessibility Engineer in a department led by the late David L. Stone (d. 1994), DEC's Vice President of Software Engineering. Stone introduced the concept of Pervasive Accessible Technology (PAT) in his address "Accessible Technology: A Look Towards the Third Wave" at the 1991 World Congress on Technology for People with Disabilities — a talk Paciello contributed to, and whose thinking grew out of work the two had done together (Paciello, 2000). Paciello carried that idea forward to the World Wide Web in 1996 (Laux et al., 1996), the lineage this paper extends. He later served as Executive Director of the Yuri Rubinsky Insight Foundation, a partner organization at the 1997 launch of the W3C Web Accessibility Initiative. He is a longtime member of the W3C, the author of Web Accessibility for People with Disabilities, and the founder of The Paciello Group (TPG), Open Access Technologies (OAT), and WebABLE.com (recently donated to Knowbility, an Austin-based accessibility non-profit organization).

References

AARP. (2022, November 10). AARP report finds that older population makes significant and growing contributions to global economy (on the *Global Longevity Economy Outlook*, research by Economist Impact). AARP.

Accessibility Standards Canada. (2024). Summary: CAN/ASC-EN 301 549:2024 Accessibility requirements for ICT products and services. Accessibility Standards Canada.

© 2026 Michael G. Paciello. All rights reserved.

Alphabet Inc. (2026). Form 10-K for the fiscal year ended December 31, 2025. U.S. Securities and Exchange Commission.

American Institutes for Research. (2026). The purchasing power of working-age adults with disabilities: Final report (April 2026). American Institutes for Research.

Apple. (2023, February 2). Apple reports first quarter results. Apple Newsroom.

Apple. (2024, September 9). macOS Sequoia Voluntary Product Accessibility Template. Apple.

Apple. (n.d.-a). Using alternative browser engines in the European Union. Apple Developer.

Apple. (n.d.-b). Using alternative browser engines in Japan. Apple Developer.

Apple. (n.d.-c). NSAccessibilityProtocol. Apple Developer Documentation.

Apple. (n.d.-d). UIAccessibility. Apple Developer Documentation.

Apple. (n.d.-e). Node sidebar in Safari Web Inspector. Safari Developer Help.

Apple. (n.d.-f). Change Advanced settings in Safari on Mac. Safari User Guide.

Apple. (n.d.-g). Use VoiceOver Recognition on your iPhone or iPad. Apple Support.

Apple Inc. (2025). Form 10-K for the fiscal year ended September 27, 2025. U.S. Securities and Exchange Commission.

Bain & Company. (2025, December 17). 2030 forecast: How agentic AI will reshape US retail. Bain & Company.

Beaudry, B. (2025, August 14). Native UI Automation for Windows in Chromium. Chrome for Developers.

Bracken, M., & Mitchell, B. (2026, April 3). White House sets $75.7B topline IT budget for fiscal 2027. FedScoop.

Chromium Project. (2013). Code review 66383005: Remove user style sheet support. Chromium Code Reviews.

Chromium Project. (2026a). extensions/common/api/_manifest_features.json, "automation" entry (main branch, read 8 October 2026). Chromium source, GitHub mirror.

Chromium Project. (2026b). ui/accessibility/OWNERS (main branch, read 8 October 2026). Chromium source, GitHub mirror.

Chromium Project. (2026c). third_party/blink/common/page/page_zoom.cc (main branch, read 8 October 2026). Chromium source, GitHub mirror.

Chromium Project. (n.d.-a). How Chrome accessibility works, part 2. Chromium source documentation.

Chromium Project. (n.d.-b). How Chrome accessibility works, part 3. Chromium source documentation.

Courthouse News Service. (2026, July 29). DOJ urges DC Circuit to beef up remedies in Google monopoly ruling. Courthouse News Service.

Deque Systems. (n.d.). Elements must meet minimum color contrast ratio thresholds (axe rule color-contrast). Deque University.

© 2026 Michael G. Paciello. All rights reserved.

Deveria, A. (2026). Can I use (data version 1.0.30001815). caniuse.com.

Duran, M. (2017, February 24). What we found when we tested tools on the world's least-accessible webpage. Accessibility in Government blog, UK Government Digital Service.

ETSI. (2021). EN 301 549 V3.2.1: Accessibility requirements for ICT products and services. ETSI.

ETSI. (2026). EN 301 549 V4.1.1: Accessibility requirements for ICT products and services. ETSI.

European Commission. (2021a). Commission Implementing Decision (EU) 2021/1339 amending Implementing Decision (EU) 2018/2048. EUR-Lex.

European Commission. (2021b). Union of Equality: Strategy for the Rights of Persons with Disabilities 2021–2030, COM(2021) 101 final. EUR-Lex.

European Commission. (n.d.-a). Harmonised standards. Single Market and Economy.

European Commission. (n.d.-b). Public procurement. Single Market and Economy.

European Parliament and Council. (2014). Directive 2014/24/EU on public procurement. EUR-Lex.

European Parliament and Council. (2019). Directive (EU) 2019/882 on the accessibility requirements for products and services. EUR-Lex.

Eurostat. (2026a). Population structure and ageing (Statistics Explained; data extracted 2 February 2026). European Commission.

Eurostat. (2026b). E-commerce statistics for individuals (Statistics Explained; data extracted February 2026). European Commission.

Fathallah, N., Hernández, D., & Staab, S. (2025). AccessGuru: Leveraging LLMs to detect and correct web accessibility violations in HTML code. In *Proceedings of the 27th International ACM SIGACCESS Conference on Computers and Accessibility (ASSETS '25)*. ACM.

Faulkner, S. (2025, January 27). Ladybird and the old-world boulevardier [Interview with Mike Smith]. HTML Accessibility.

Faulkner, S. (Ed.). (2026). Screen reader HTML support tables [Blog post and lookup tool]. TetraLogical. Blog posted 10 July 2025, updated 21 August 2026; lookup tool updated 7 May 2026. Retrieved from blog post and lookup tool.

Federal Acquisition Regulation. (n.d.). FAR 39.203: Applicability (read 9 October 2026). Acquisition.gov.

Federal Register. (2026). Documents from the Architectural and Transportation Barriers Compliance Board published since January 2024 (search, read 8 October 2026). Office of the Federal Register.

Google. (2024, September 24). Frictionless PDF accessibility with ML and OCR. ChromeOS.dev.

Google. (2025). Chrome 141 release notes. Chrome for Developers.

Google. (2026a). Accessibility features reference, Chrome DevTools. Chrome for Developers.

Google. (2026b, June 16). Expanded availability: Gemini in Chrome. Google Workspace Updates.

© 2026 Michael G. Paciello. All rights reserved.

Google. (2026c, August 11). More than 1 billion people are using the Gemini app every month. The Keyword, Google.

Google. (n.d.-a). chrome.scripting API reference. Chrome for Developers.

Google. (n.d.-b). Get image descriptions on Chrome. Google Chrome Help.

Google. (n.d.-c). Manage captions and translations in Chrome. Google Chrome Help.

Google. (n.d.-d). Use Reading mode in Chrome. Google Chrome Help.

Google. (n.d.-e). Use a text cursor to navigate and select text. Google Accessibility Help.

Google. (n.d.-f). Accessibility for customers and partners (Accessibility Conformance Reports). Google.

Google. (n.d.-g). Chrome DevTools MCP tool reference (read 8 October 2026). GitHub.

Google. (n.d.-h). Chrome Enterprise Premium (read 8 October 2026). Google.

Google. (n.d.-i). Google AI plans (read 9 October 2026). Google One.

Google. (n.d.-j). Prompt API. Chrome for Developers documentation (page last updated 26 August 2026; read 16 October 2026).

Hecker, F. (2007, November 19). Mozilla Foundation grants and related expenditures for 2007. frankhecker.com.

Holmberg, C., & Hellström, G. (2021). T.140 Real-Time Text Conversation over WebRTC Data Channels (RFC 8865). RFC Editor.

HTTP Archive. (2021). Web Almanac 2021: Third parties. HTTP Archive.

Jauhiainen, T., Lui, M., Zampieri, M., Baldwin, T., & Lindén, K. (2019). Automatic language identification in texts: A survey. *Journal of Artificial Intelligence Research*, *65*, 675–782.

Koenecke, A., Nam, A., Lake, E., Nudell, J., Quartey, M., Mengesha, Z., Toups, C., Rickford, J. R., Jurafsky, D., & Goel, S. (2020). Racial disparities in automated speech recognition. *Proceedings of the National Academy of Sciences*, *117*(14), 7684–7689.

Kulikowski, K., & Hablich, M. (2026, June 22). A developer toolkit to make your website agent-ready. Chrome for Developers.

Kulikowski, K., & More, O. (2026). Build agent-friendly websites (last updated 1 April 2026). web.dev.

Ladybird Browser Initiative. (2026). Add screen-reader support: Orca and VoiceOver (pull request #9156). GitHub.

Ladybird Browser Initiative. (n.d.). Ladybird project website. Ladybird Browser Initiative.

Laux, L. F., McNally, P. R., Paciello, M. G., & Vanderheiden, G. C. (1996). Designing the World Wide Web for people with disabilities: A user centered design approach. In *Proceedings of the Second Annual ACM Conference on Assistive Technologies (ASSETS '96)* https://doi.org/10.1145/228347.228363

Legal Information Institute. (n.d.). 29 U.S. Code § 794d: Electronic and information technology. Cornell Law School.

© 2026 Michael G. Paciello. All rights reserved.

Levels.fyi. (2025). End of year pay report 2025. Levels.fyi.

Leventhal, A. (2001a, January 5). Message on Netscape accessibility work to the WAI Interest Group. W3C mailing list archive.

Leventhal, A. (2001b, July 10). Message on MSAA support in Mozilla to the WAI User Agent Working Group. W3C mailing list archive.

Leventhal, A. (2001c, December 17). Mozilla accessibility alpha announcement. W3C mailing list archive.

Li, C., Padmanabhuni, S., Cheema, M., Seifi, H., & Fazli, P. (2025). VideoA11y: Method and dataset for accessible video description. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI '25)*. ACM.

Linux Foundation. (n.d.). IAccessible2. Linux Foundation wiki.

Linux Foundation Europe. (2023, September 7). Servo web rendering engine joins Linux Foundation Europe. Linux Foundation Europe.

MacLeod, H., Bennett, C. L., Morris, M. R., & Cutrell, E. (2017). Understanding blind people's experiences with computer-generated captions of social media images. In *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems (CHI '17)*. ACM.

McKinsey & Company. (2025, October 17). The agentic commerce opportunity: How AI agents are ushering in a new era for consumers and merchants. McKinsey & Company.

MDN. (2026). browser-compat-data, version 8.1.5. GitHub.

MediaPost. (2026, August 7). Tougher Google antitrust penalties threaten Firefox, Mozilla warns. MediaPost.

Microsoft. (2022, March 17). "Appears to say": Microsoft Edge auto-generated image labels. Microsoft Edge Blog.

Microsoft. (2025a, June 30). Removing -ms-high-contrast and embracing standards-based forced colors in Microsoft Edge. Microsoft Edge Blog.

Microsoft. (2025b, October 1). Meet Microsoft 365 Premium: Your AI and productivity powerhouse. Microsoft 365 Blog.

Microsoft. (2026a). Accessibility features in Microsoft Edge. Microsoft Support.

Microsoft. (2026b). LiveCaptionsAllowed policy. Microsoft Learn.

Microsoft. (2026c). Use Reading mode in Microsoft Edge. Microsoft Support.

Microsoft. (2026d, May 13). New updates to Edge across desktop and mobile. Microsoft Edge Blog.

Microsoft. (n.d.-a). Microsoft Active Accessibility and UI Automation compared. Microsoft Learn.

Microsoft. (n.d.-b). Reading mode in Microsoft Edge (feature page). Microsoft.

Microsoft. (n.d.-c). Playwright MCP server README (read 8 October 2026). GitHub.

Microsoft. (n.d.-d). Microsoft Edge for Business (read 8 October 2026). Microsoft.

© 2026 Michael G. Paciello. All rights reserved.

Microsoft Corporation. (2026a). Fiscal year 2026 fourth quarter earnings release (Form 8-K, Exhibit 99.1). U.S. Securities and Exchange Commission.

Microsoft Corporation. (2026b, January 28). Earnings conference call, fiscal year 2026 second quarter. Microsoft Investor Relations.

Microsoft Edge. (n.d.). ARIA Notify explainer. GitHub.

Mohanbabu, A. G., & Pavel, A. (2024). Context-aware image descriptions for web accessibility. In *Proceedings of the 26th International ACM SIGACCESS Conference on Computers and Accessibility (ASSETS '24)*. ACM.

Morgan Stanley. (2025, December 8). Here come the shopping bots. Morgan Stanley.

Mozilla. (2024a, November 4). Help us improve our alt text generation model. Mozilla Blog.

Mozilla. (2024b). Firefox VPAT 124 on Windows (April 2024). Mozilla wiki.

Mozilla. (2026a). Firefox 150 for developers. MDN Web Docs.

Mozilla. (2026b). modules/libpref/init/StaticPrefList.yaml, release branch (read 8 October 2026). Firefox source, GitHub mirror.

Mozilla. (n.d.-a). Firefox accessibility architecture. Firefox Source Docs.

Mozilla. (n.d.-b). Mozilla accessibility on Unix: architecture. Mozilla archive.

Mozilla. (n.d.-c). Accessibility Inspector. Firefox Source Docs.

Mozilla. (n.d.-d). Reader View. Mozilla Support.

Mozilla. (n.d.-e). Make reading easier with Firefox's contrast control settings. Mozilla Support.

Mozilla. (n.d.-f). Font size and zoom. Mozilla Support.

Mozilla. (n.d.-g). Access AI chatbots in Firefox. Mozilla Support.

Mozilla. (n.d.-h). Firefox Public Data Report: User activity, worldwide monthly active users (data file read 8 October 2026). Mozilla.

Mozilla. (n.d.-i). Prompt API [Standards position, mozilla/standards-positions issue #1213] (read 16 October 2026). GitHub.

Mozilla Accessibility Team. (2023, May 9). Firefox 113 significantly boosts accessibility performance. Mozilla Accessibility Blog.

Mozilla Foundation. (2025). Consolidated financial statements, 2024 and 2023. State of Mozilla.

NLnet. (n.d.). WPT automatic testing for platform accessibility mappings. NLnet Foundation.

NSW Government. (2023). Government accessibility standards for ICT. buy.nsw.

NV Access. (2025). NVDA issue #19276, including a maintainer comment comparing Chromium's UIA and IA2 implementations. GitHub.

Office of Management and Budget. (2023, December 21). M-24-08: Strengthening digital accessibility and the management of Section 508 of the Rehabilitation Act. Executive Office of the President.

© 2026 Michael G. Paciello. All rights reserved.

OpenAI. (2026, February 27). Scaling AI for everyone. OpenAI.

OpenAI. (n.d.-a). Publishers and developers: FAQ (read 8 October 2026). OpenAI Help Center.

OpenAI. (n.d.-b). Evolving Atlas into ChatGPT for browser-based agentic work (read 8 October 2026). OpenAI Help Center.

Opera. (2025, December 11). Opera opens public access to Opera Neon, its experimental agentic AI browser. Opera press.

Oppermann, C. (1997, December 5). Message on Microsoft Active Accessibility 1.1 to the WAI Interest Group. W3C mailing list archive.

Pew Research Center. (2021, September 10). Americans with disabilities less likely than those without to own some digital devices. Pew Research Center.

Pew Research Center. (n.d.). Internet use by age (2025 data; read 9 October 2026). Pew Research Center.

PowerMapper Software. (2025, December 14). Screen reader compatibility. PowerMapper.

Paciello, M. (2000). Web Accessibility for People with Disabilities. CMP Books.

Servo Project. (2026a, March 31). February in Servo. servo.org.

Servo Project. (2026b, August 31). July in Servo. servo.org.

Servo Project. (2026c, April 13). Servo is now available on crates.io. servo.org.

Servo Project. (n.d.-a). Sponsorship. servo.org.

Servo Project. (n.d.-b). Accessibility, The Servo Book. servo.org.

Simmons, J. (2024, February 1). The web just gets better with Interop 2024. WebKit Blog.

Simmons, J. (2025, January 14). The success of Interop 2024! WebKit Blog.

Sovereign Tech Agency. (n.d.). Servo. Sovereign Tech Agency.

Stack Overflow. (2025). 2025 Developer Survey: Work. Stack Overflow.

StatCounter. (2026). Browser market share worldwide, September 2026. StatCounter Global Stats.

Stone, D. (1991). Accessible Technology: A Look Towards the Third Wave [Conference address]. World Congress on Technology for People with Disabilities. As documented in Paciello (2000), Web Accessibility for People with Disabilities (Chapter 11, pp. 272–273).

Terrell, K. (2019, December 19). Americans 50 and older would be world's third-largest economy, AARP study finds. AARP.

United States v. Google LLC, No. 1:20-cv-03010-APM (D.D.C. Sept. 2, 2025; memorandum opinion and final judgment, Dec. 5, 2025). U.S. Department of Justice.

Updegrove, A. (2006, December 14). Showing the accessibility way: IBM contributes Project Missouri to the Free Standards Group. ConsortiumInfo.org.

© 2026 Michael G. Paciello. All rights reserved.

U.S. Access Board. (2017). Information and Communication Technology (ICT) Standards and Guidelines, 36 CFR Parts 1193 and 1194. U.S. Access Board.

U.S. Bureau of Labor Statistics. (2026, September 9). Employer costs for employee compensation, June 2026. U.S. Department of Labor.

U.S. Census Bureau. (2024). Disability status by age group (American Community Survey, 2022). U.S. Census Bureau.

U.S. Census Bureau. (2026, June 25). Populations in all age groups growing in the South, driven by outlying counties in metro areas (press release CB26-105). U.S. Census Bureau.

U.S. Department of Justice. (2026a, April 20). Extension of compliance dates for nondiscrimination on the basis of disability; accessibility of web information and services of state and local government entities (interim final rule). Federal Register.

U.S. Department of Justice. (2026b, July 28). Response brief and opening brief on cross-appeal for the United States and co-plaintiff states (page proof), United States v. Google LLC, D.C. Cir. Nos. 26-5023, 26-5047, 26-5049. U.S. Department of Justice.

U.S. Department of Justice & U.S. Department of Education. (2010, June 29). Joint "Dear Colleague" letter on electronic book readers. U.S. Department of Justice ADA archive.

U.S. General Services Administration. (2024, December). Governmentwide findings: Acquisition and procurement (FY2024 Section 508 assessment). Section508.gov.

U.S. General Services Administration. (2026, March). FY2025 government-wide Section 508 assessment: Executive summary. Section508.gov.

U.S. General Services Administration. (n.d.-a). Buy accessible (read 8 October 2026). Section508.gov.

U.S. General Services Administration. (n.d.-b). ACR/VPAT FAQ (read 9 October 2026). Section508.gov.

W3C. (2015). User Agent Accessibility Guidelines (UAAG) 2.0 (W3C Working Group Note, 15 December 2015). W3C.

W3C. (2023). Accessible Rich Internet Applications (WAI-ARIA) 1.2 (W3C Recommendation, 6 June 2023). W3C.

W3C. (2024). Web Content Accessibility Guidelines (WCAG) 2.2 (W3C Recommendation, 12 December 2024). W3C.

W3C. (2026a). Core Accessibility API Mappings 1.2 (Candidate Recommendation Draft, 23 September 2026). W3C.

W3C. (2026b). Accessible Rich Internet Applications (WAI-ARIA) 1.3 (Working Draft). W3C.

W3C. (2026c). Using ARIA (Discontinued Draft, 24 February 2026). W3C.

W3C. (2026d). Web Authentication: An API for accessing Public Key Credentials, Level 3 (W3C Recommendation, 25 August 2026). W3C.

W3C. (2026e). HTML Accessibility API Mappings 1.0 (Working Draft, 5 October 2026). W3C.

W3C. (2026f). Accessible Name and Description Computation 1.2 (Working Draft, 2 October 2026). W3C.

© 2026 Michael G. Paciello. All rights reserved.

W3C. (n.d.-a). WAI-Adapt Explainer: publication history. W3C.

W3C. (n.d.-b). WAI-Adapt: Symbols Module: publication history. W3C.

W3C Accessibility at the Edge Community Group. (2025). Accessibility capabilities: Post-source code and content (Final Community Group Report, 4 November 2025; not a W3C Standard). Sajka, J. (Ed.); Sajka, J., Wolberger, L., Taylor, J., & Paciello, M. (Authors). The author of this paper is a co-author of this report.

W3C ARIA and Assistive Technologies Community Group. (2026). ARIA-AT reports (read 8 October 2026). W3C.

W3C TAG. (n.d.-a). Prompt API [Design review, w3ctag/design-reviews issue #1093] (read 16 October 2026). GitHub.

W3C WAI. (2024a). User Agent Accessibility Guidelines (UAAG) overview. W3C Web Accessibility Initiative.

W3C WAI. (2024b, September 17). Captions/subtitles, Making Audio and Video Media Accessible. W3C Web Accessibility Initiative.

W3C WAI. (n.d.-a). Read me first, ARIA Authoring Practices Guide. W3C Web Accessibility Initiative.

W3C WAI. (n.d.-b). Treegrid pattern, ARIA Authoring Practices Guide. W3C Web Accessibility Initiative.

W3C WAI. (n.d.-c). Understanding Success Criterion 1.4.12: Text spacing. W3C Web Accessibility Initiative.

Web Platform Tests. (2026). Aligned stable results for accname, wai-aria, html-aam, and core-aam (runs of 6 October 2026; the link shows the latest runs). wpt.fyi.

Web Platform Tests. (n.d.-a). Interop 2024 dashboard. wpt.fyi.

Web Platform Tests. (n.d.-b). aamtest tests. web-platform-tests.org.

Web Platform Tests. (n.d.-c). interop-accessibility repository README. GitHub.

WebAIM. (2026). The WebAIM Million: The 2026 report on the accessibility of the top 1,000,000 home pages. WebAIM.

WebKit. (n.d.-a). ML Prompt API [Standards position, WebKit/standards-positions issue #495] (read 16 October 2026). GitHub.

WebKit Bugzilla. (2019). Bug 193348: AX: Introduce isolated accessibility tree. WebKit Bugzilla.

WebKit Bugzilla. (2022). Bug 230253: Use the AT-SPI D-Bus API instead of ATK. WebKit Bugzilla.

WebKit Bugzilla. (2025). Bug 299705: isolated accessibility tree work, fixed 7 October 2025. WebKit Bugzilla.

WebKit Project. (2022). Web Inspector reference: Audit tab. WebKit.

WebKit Project. (2026). Source/WebCore/page/CaptionUserPreferencesMediaAF.cpp (main branch, read 8 October 2026). WebKit source, GitHub mirror.

© 2026 Michael G. Paciello. All rights reserved.

World Bank. (2026). Population ages 65 and above, total and percent of total population, World (indicators SP.POP.65UP.TO and SP.POP.65UP.TO.ZS) (last updated 13 July 2026; based on UN World Population Prospects). World Bank.

World Health Organization. (2021). World report on hearing. WHO.

World Health Organization. (2022). Global report on health equity for persons with disabilities. WHO.

World Health Organization. (2026, February 10). Blindness and vision impairment (fact sheet). WHO.

Xue, T., Qi, W., Shi, T., Song, C. H., Gou, B., Song, D., Sun, H., & Su, Y. (2025). An illusion of progress? Assessing the current state of web agents (arXiv:2504.01382). arXiv.

Zecchini, G. (2026, September 29). How AI agents use websites, where they fail, and what to fix. Merj.

Zhou, S., Xu, F. F., Zhu, H., Zhou, X., Lo, R., Sridhar, A., Cheng, X., Ou, T., Bisk, Y., Fried, D., Alon, U., & Neubig, G. (2023). WebArena: A realistic web environment for building autonomous agents (arXiv:2307.13854). arXiv.

© 2026 Michael G. Paciello. All rights reserved.